UNIT – 5
Exploring graphs

COMING UP!!

- An introduction using graphs
- Traversing Trees –Preconditioning
- Depth First Search -Undirected Graph
- Depth First Search -Directed Graph
- Topological sorting
- Breadth First Search
- Backtracking
- The general template
- The Knapsack Problem using backtracking
- The Eight queens problem

An introduction using graphs

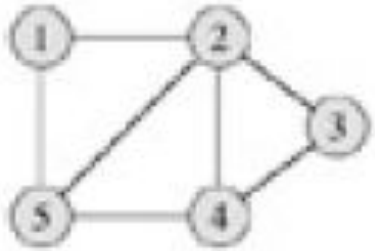
- There are two standard ways to represent a graph G = (V, E): as a collection of adjacency lists or as an adjacency matrix. Either way is applicable to both directed and undirected graphs.

- The adjacency-list representation is usually preferred, because it provides a compact way to represent sparse graphs-those for which |E| is much less than |V| ∧ 2.

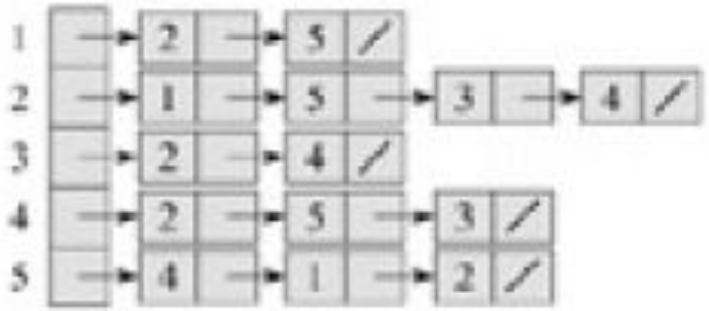
- An adjacency-matrix representation may be preferred, however, when the graph is dense-|E| is close to |V| ∧ 2-or when we need to be able to tell quickly if there is an edge.

An introduction using graphs and games

- Two representations of an undirected graph. (a) An undirected graph G having five vertices and seven edges. (b) An adjacency-list representation of G. (c) The adjacency matrix representation of G



(a)



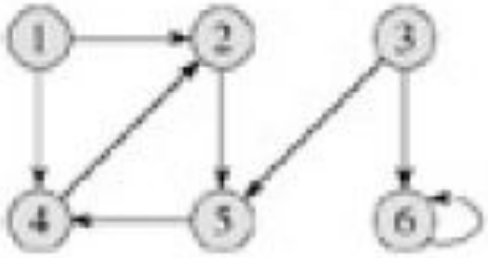
(b)

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

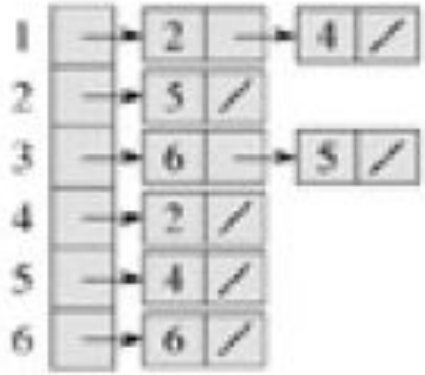
(c)

An introduction using graphs and games

- Two representations of a directed graph. (a) A directed graph G having six vertices and eight edges. (b) An adjacency-list representation of G. (c) The adjacency-matrix representation of G.



(a)



(b)

	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

(c)

Traversing Trees – Preconditioning

- We all know about Tree and Binary Tree
- We can traverse Binary Tree in following manners
  - Preorder - Root □ Left □ Right
  - Inorder – Left □ Root □ Right
  - Postorder – Left □ Right □ Root
- Preorder and Postorder generalize in an obvious way to non binary trees.
- These three techniques explore the tree from left to right. Three corresponding techniques explore the tree from right to left.
- Implementation of any of these techniques using recursion is straightforward.

Traversing Trees – Preconditioning

- **Preconditioning** –
  - If we have to solve several similar instances of the same problem, it may be worthwhile to invest some time in calculating auxiliary results that can thereafter be used to speed up the solution of each instance. **This is Preconditioning.**
  - Informally, let **a** be the time it takes to solve a typical instance when no auxiliary information is available, let **b** the time it takes to solve a typical instance when auxiliary information is available, and let **p** be the time needed to calculate this extra information.

Traversing Trees – Preconditioning

- **Preconditioning** –
  - To solve n typical instances takes time **na** without preconditioning and time **p + nb** if preconditioning is used.
  - Provided **b < a,** it is advantageous to use preconditioning when **n > p/(a-b).**
  - From this point of view, the dynamic programming algorithm for making change gives an example of preconditioning.
  - Once the necessary values have been calculated, we can make change quickly whenever this is required.

Traversing Trees – Preconditioning

- **Preconditioning** –
  - Even when there are few instances to be solved, pre computation of auxiliary information may be useful.
  - Suppose we occasionally have to solve one instance from some large set of possible instances.
  - When a solution is needed it must be provided very rapidly, for example to ensure sufficiently fast response for a real time application.

Traversing Trees – Preconditioning

- **Preconditioning** –
  - As a second example of this technique we use the problem of determining **ancestry in a rooted tree**. Let T be a rooted tree, not necessarily binary.
  - We say that a node **v** of **T** is an ancestor of node **w** if **v** lies on the path from **w** to the root of **T**.
  - In particular, every node is its own ancestor and the root is an ancestor of every node.

Traversing Trees – Preconditioning

- **Preconditioning** –
  - The problem is thus, given a pair of nodes **(v, w)** from T, to determine whether or not **v** is an ancestor of **w**.
  - If T contains n nodes, any direct solution of this instance takes a time in O(n).
  - However it is possible to precondition T in a time of O(n) so we can subsequently solve any particular instance of the ancestry problem in constant time.

Traversing Trees – Preconditioning

- **Preconditioning** –
  - To precondition the tree, we traverse it first in preorder and then in postorder, numbering the nodes sequentially as we visit them.
  - For a node v, let prenum[v] be the number assigned to v when we traverse the tree in preorder, and let postnum[v] be the number assigned during the traversal in postorder.

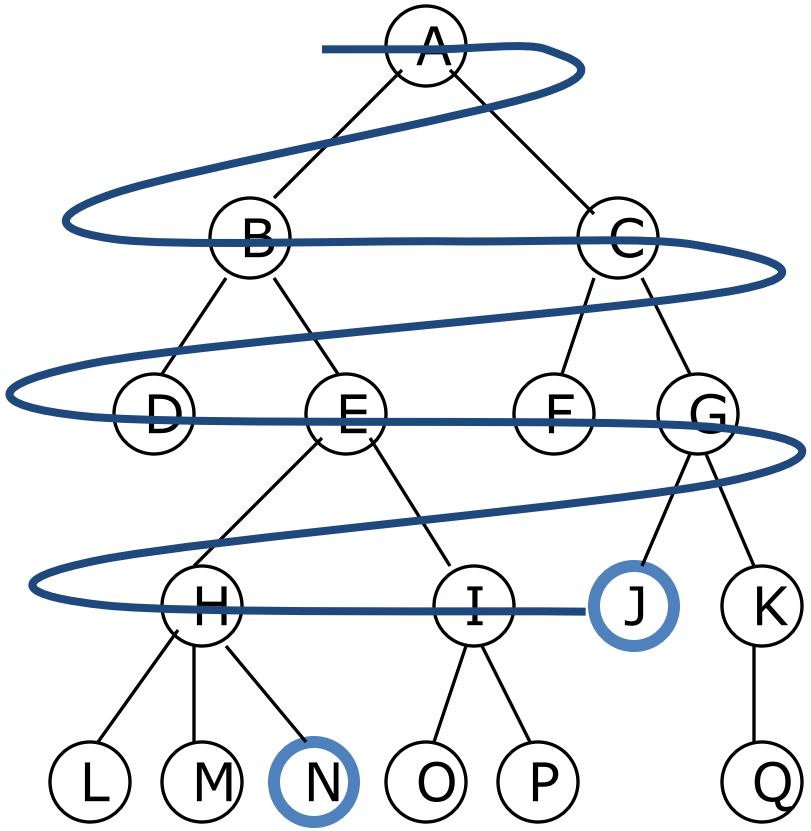
Traversing Trees – Preconditioning

- **Preconditioning** –
  - Let v and w be two nodes in the tree. In preorder we first number a node and then number its subtress from left to right. Thus
    - If prenum[v] <= prenum[w] then v is an ancestor of w **OR** v is to the left of w in the tree.
  - In postorder we first number the subtrees of a node from left to right, and then we number the node itself. Thus
    - If postnum[v] >= postnum[w] then v is an ancestor of w OR v is to the right of w in the tree.
  - It follows that
    - If prenum[v] <= prenum[w] and postnum[v] >= postnum[w] then v is an ancestor of w.
    - Once the values of prenum and postnum have been calculated in a time in $\Theta(n)$, the required condition can be checked in a time in $\Theta(1)$.

Breadth-First Search

- "Explore" a graph, turning it into a tree
  - One vertex at a time
  - Expand frontier of explored vertices across the *breadth* of the frontier
- Builds a tree over the graph
  - Pick a *source vertex* to be the root
  - Find ("discover") its children, then their children, etc.

Breadth-first searching

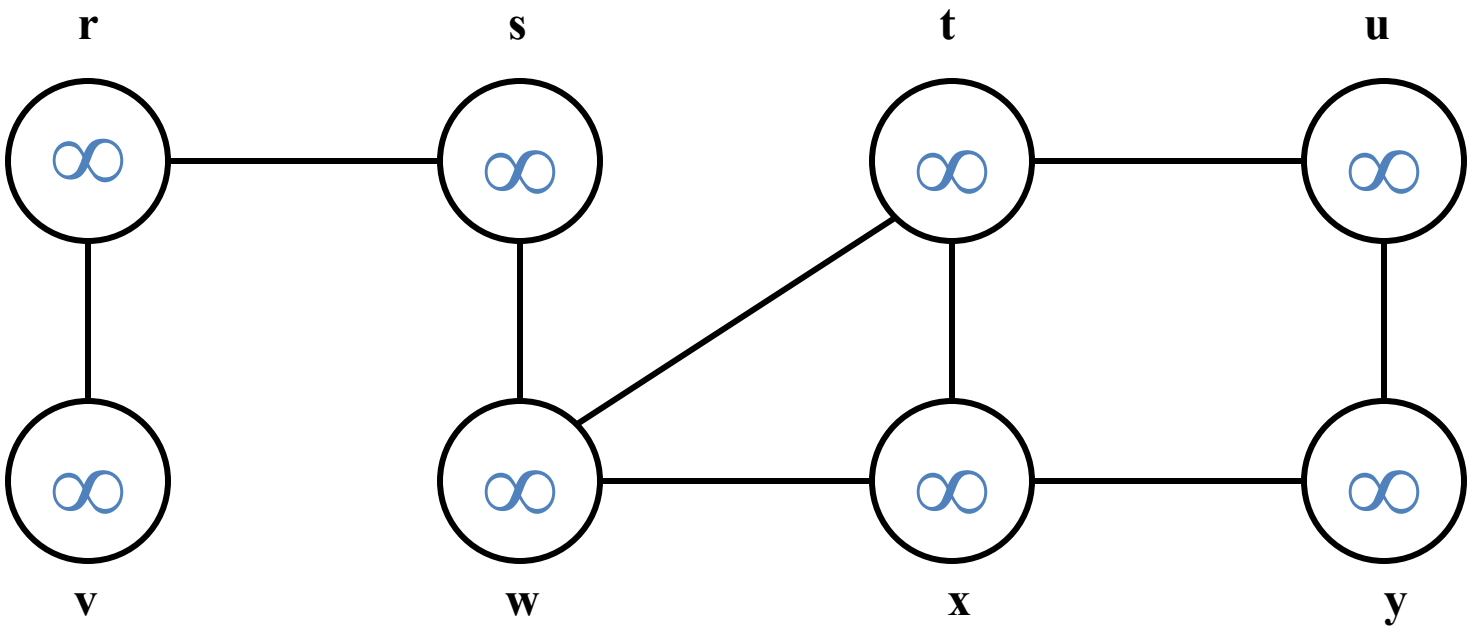


- A breadth-first search (BFS) explores nodes nearest the root before exploring nodes further away
- For example, after searching A, then B, then C, the search proceeds with D, E, F, G
- Node are explored in the order A B C D E F G H I J K L M N O P Q
- J will be found before N

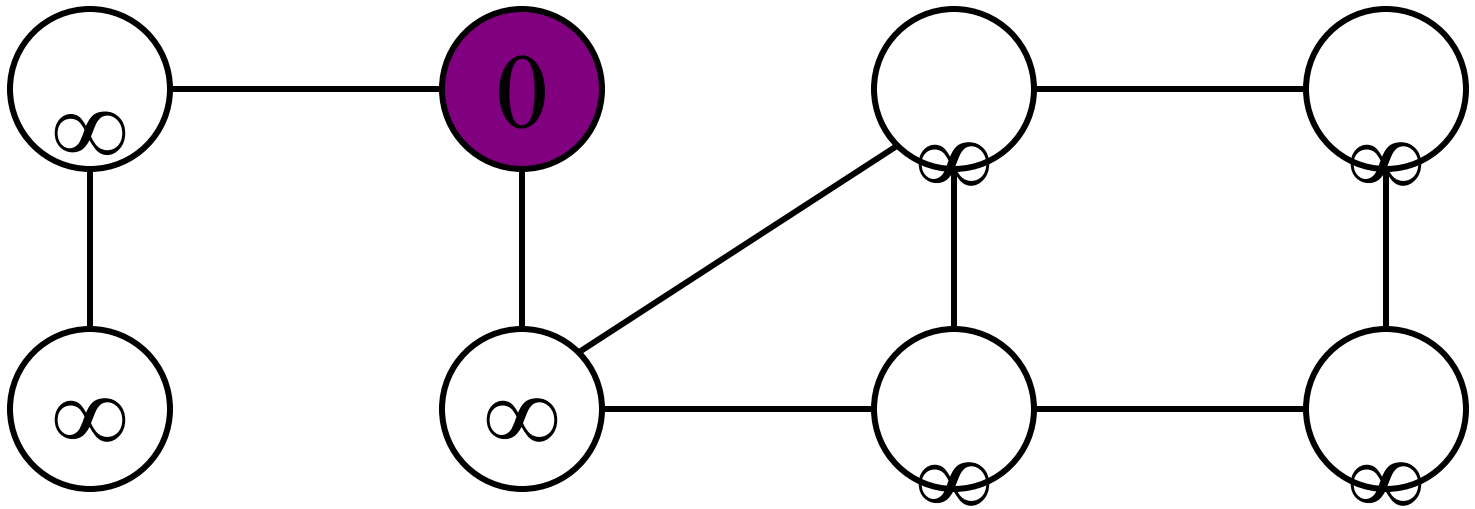
Breadth-First Search

- Again will associate vertex “colors” to guide the algorithm
  - White vertices have not been discovered
    - All vertices start out white
  - Grey vertices are discovered but not fully explored
    - They may be adjacent to white vertices
  - Black vertices are discovered and fully explored
    - They are adjacent only to black and gray vertices
- Explore vertices by scanning adjacency list of grey vertices

Breadth-First Search: Example

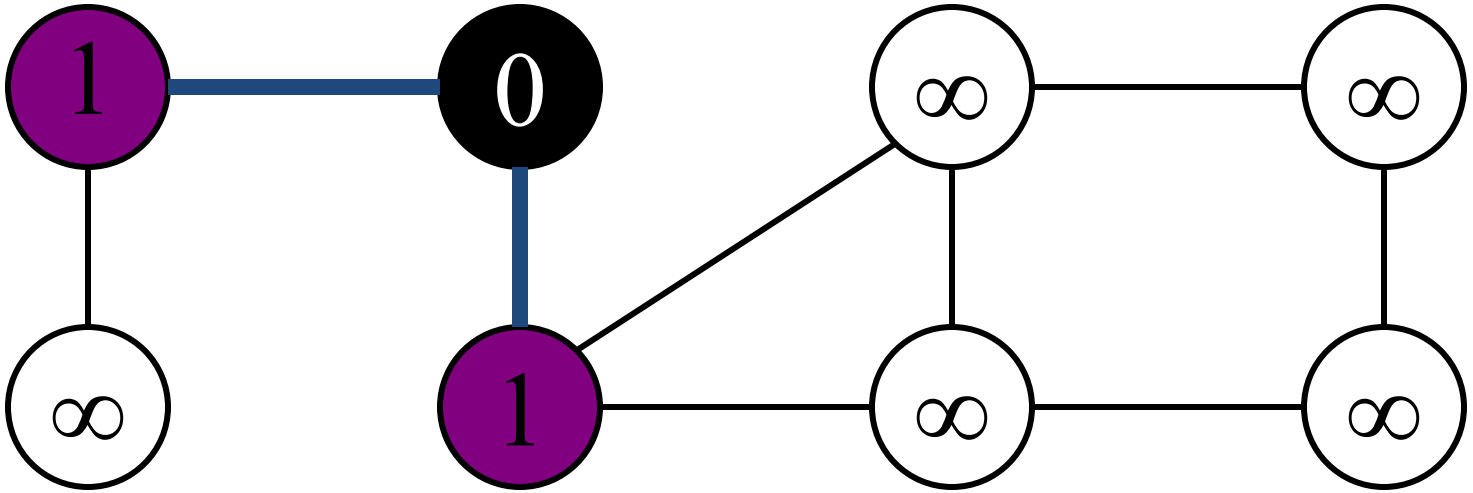


Breadth First Search



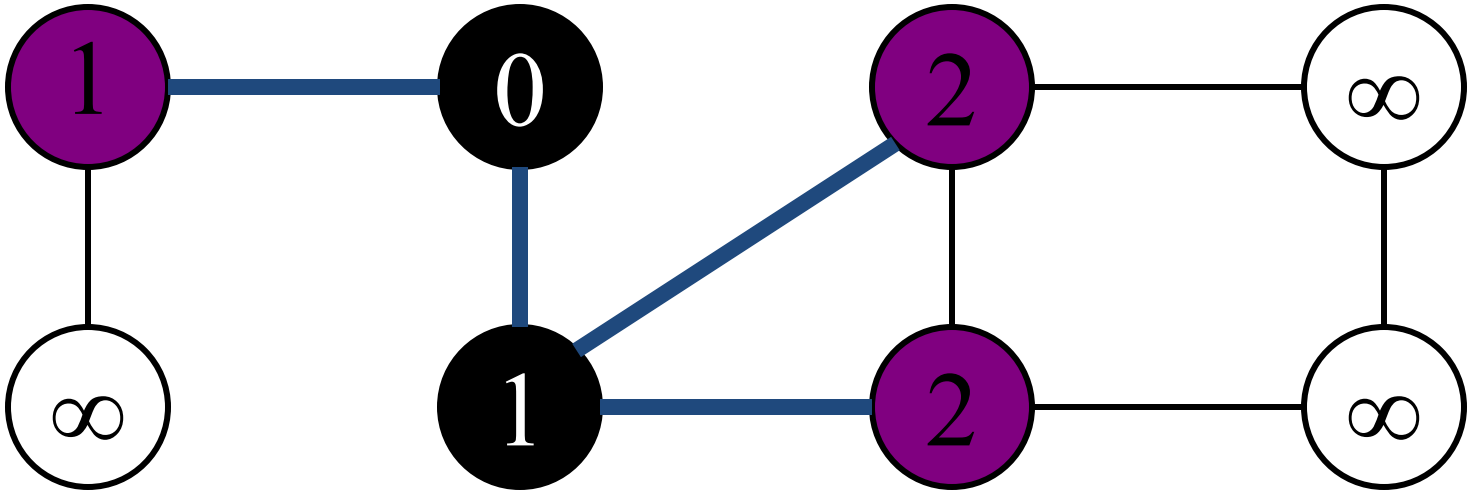
Q: S

Breadth First Search



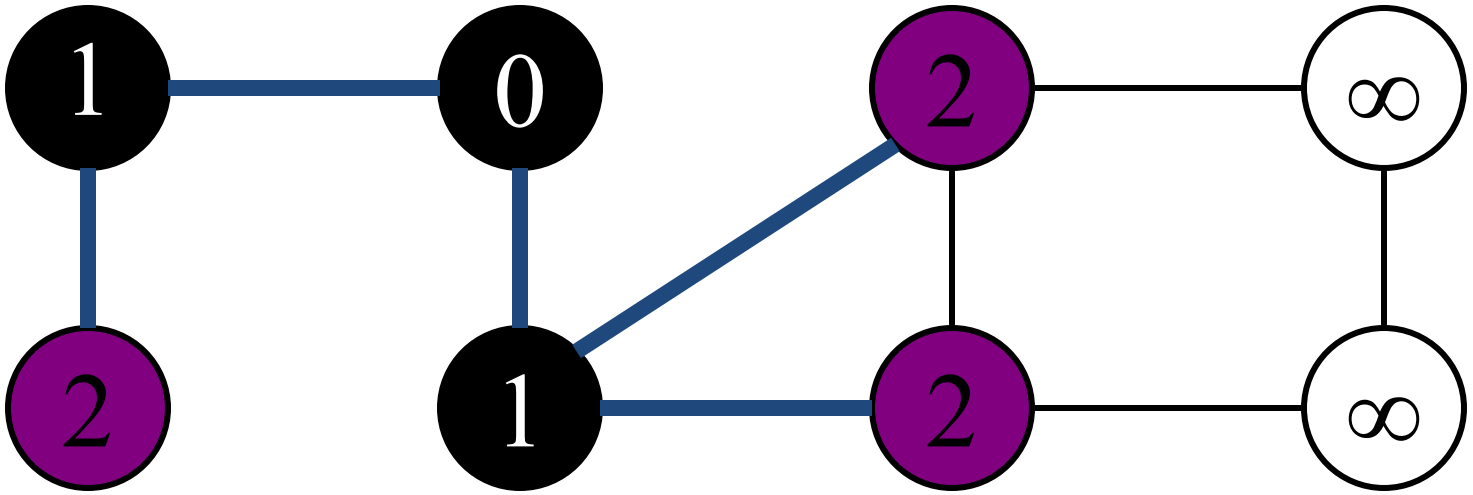
Q: | **w** | **r** |

Breadth First Search



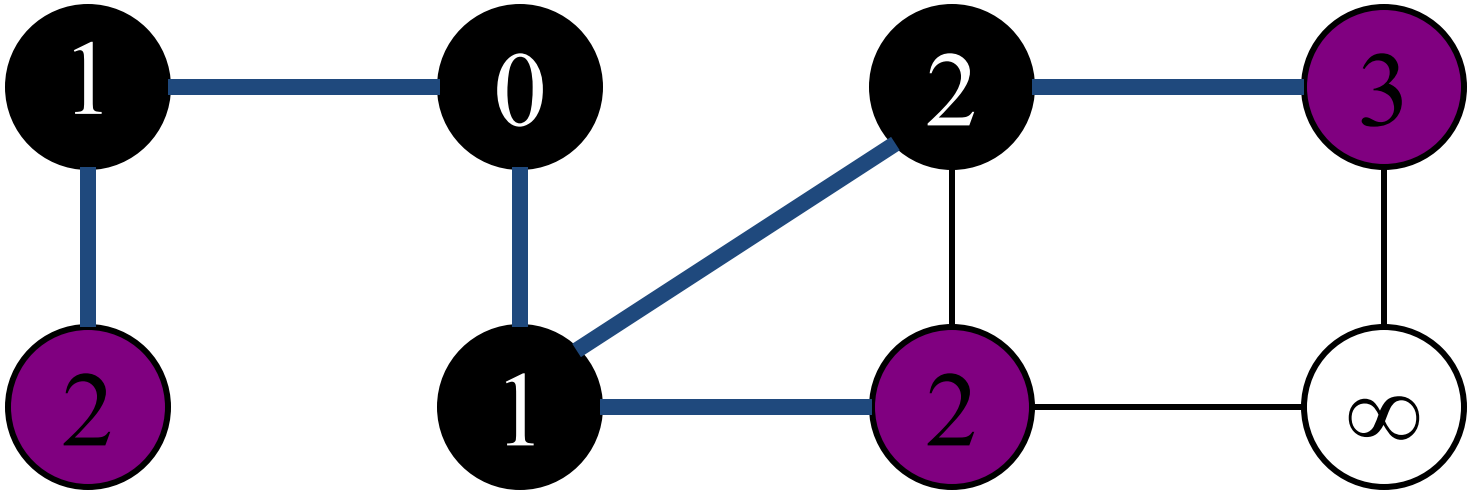
Q: | r | t | x |

Breadth First Search



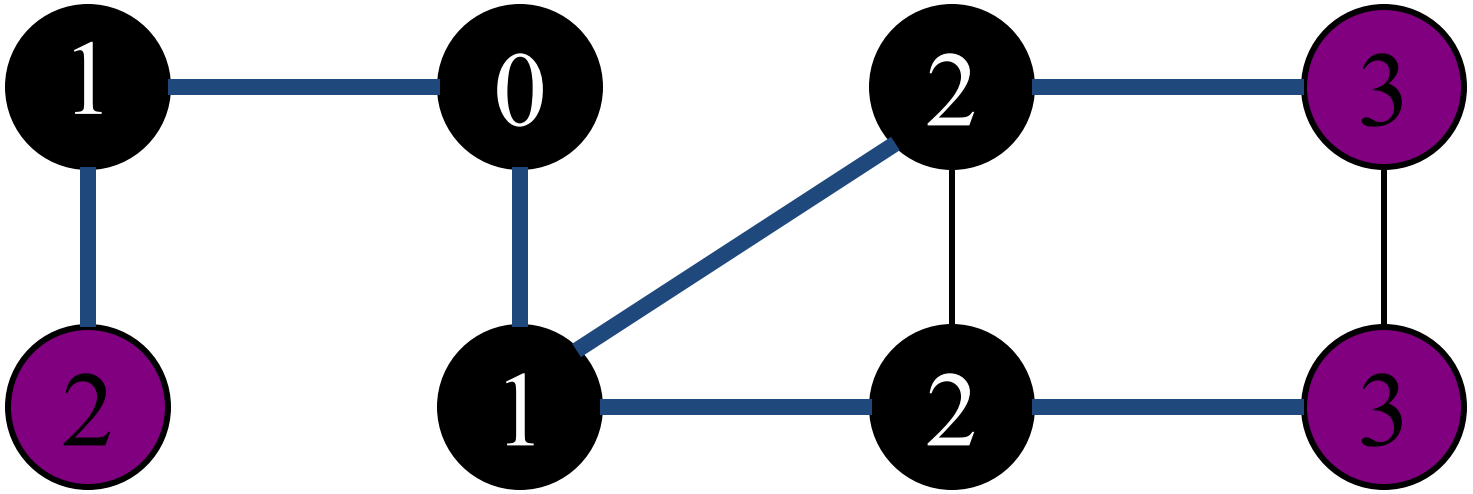
Q: | t | x | v |

Breadth First Search



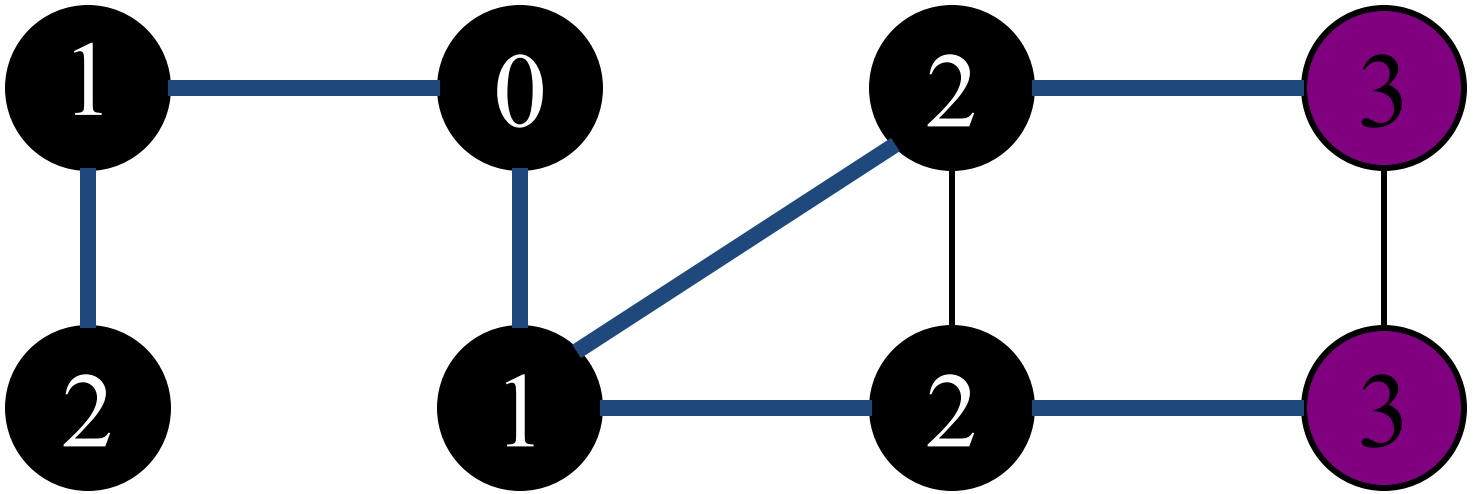
Q: | x | v | u |

Breadth First Search



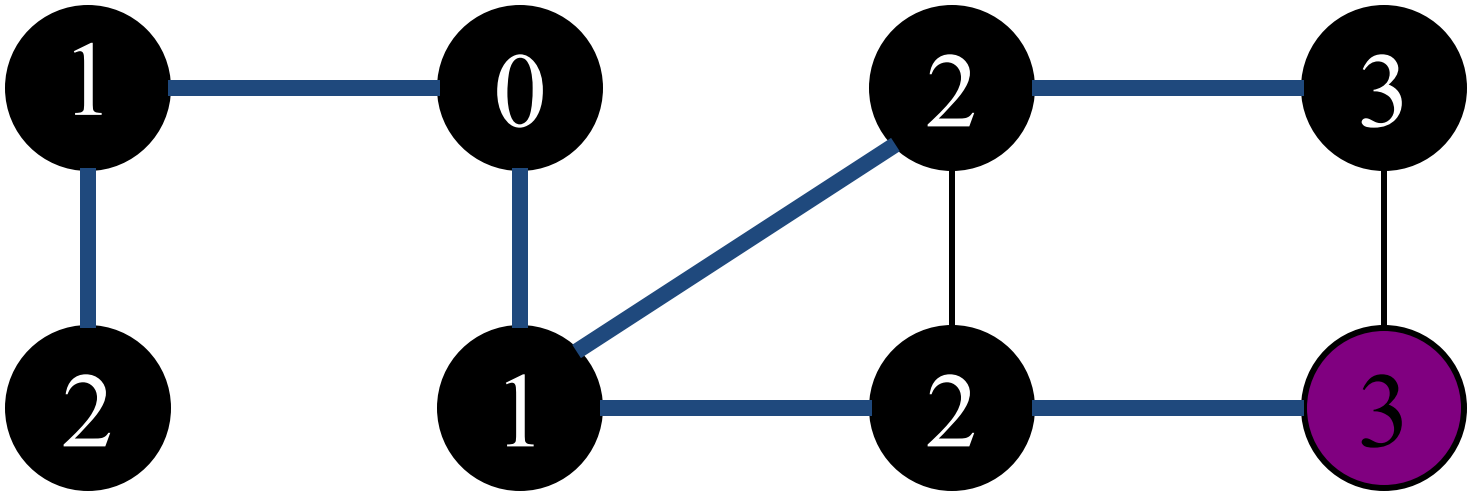
Q: | v | u | y |

Breadth First Search



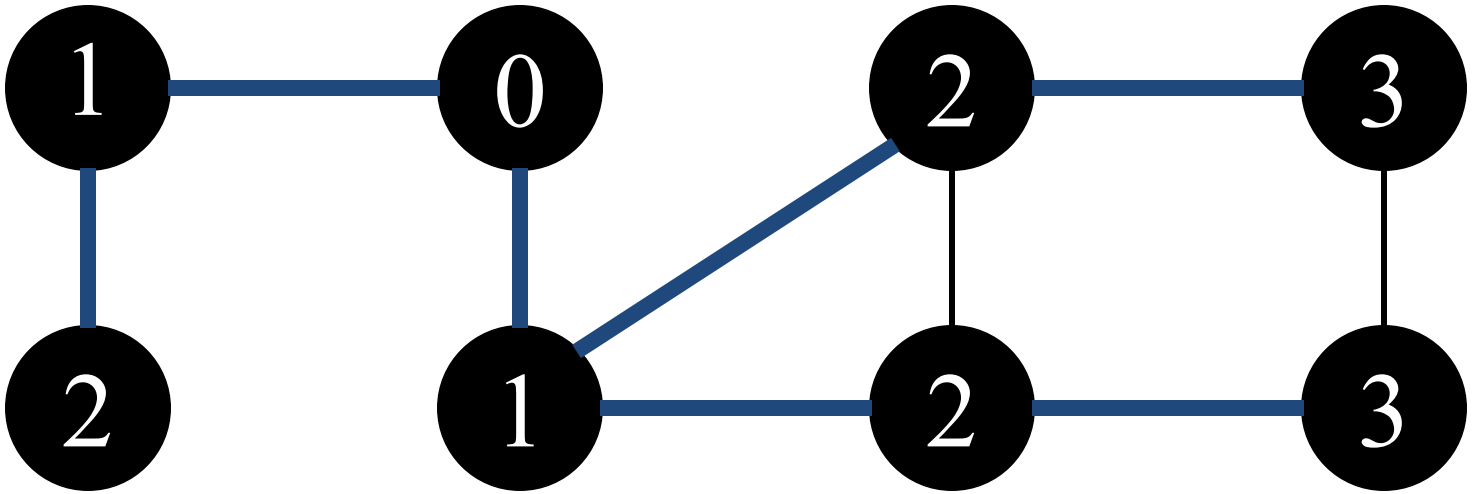
Q: | **u** | **y** |

Breadth First Search



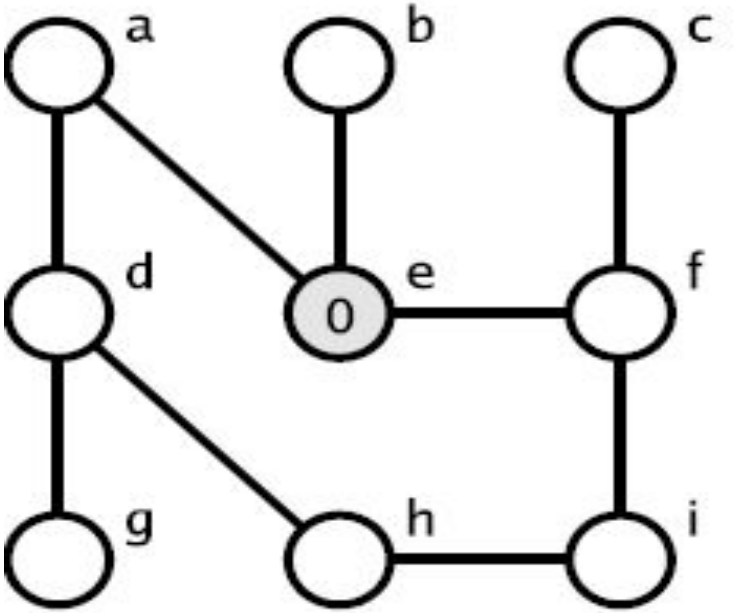
Q: y

Breadth First Search



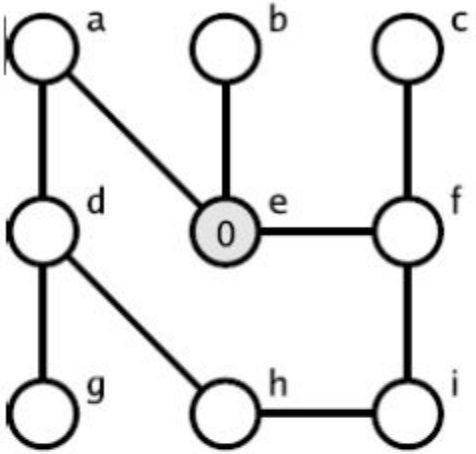
Ø

Example using BFS

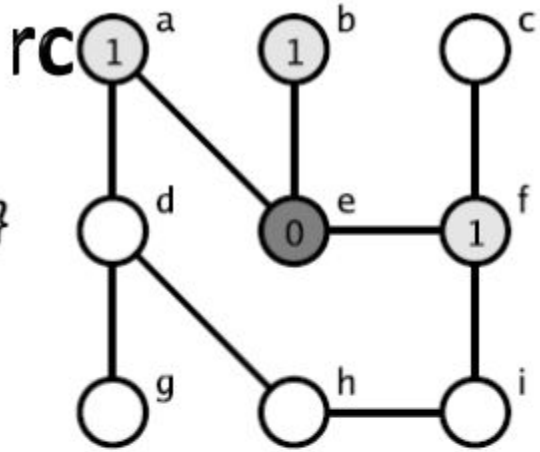


$Q = \{e\}$

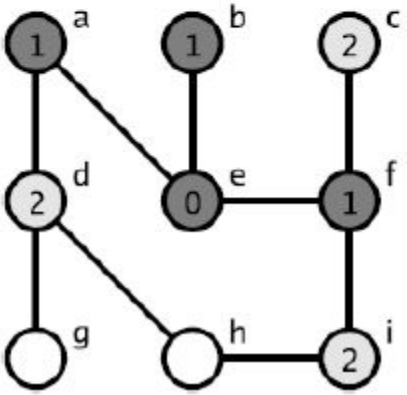
Example



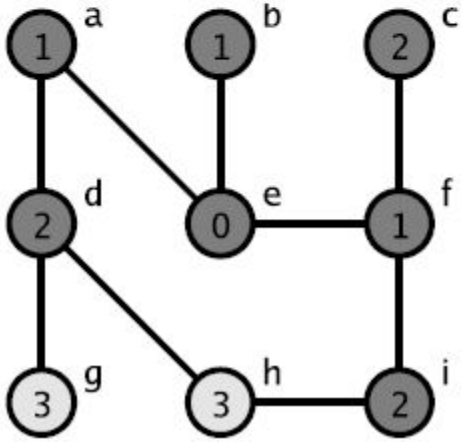
$Q = \{e\}$



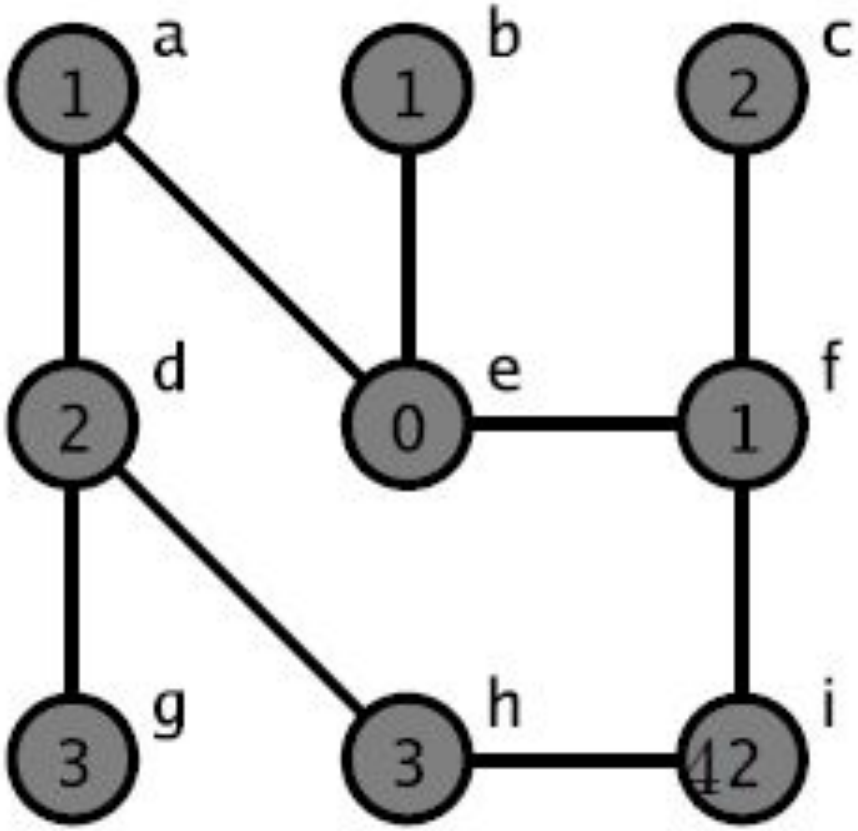
$Q = \{a,b,f\}$



$Q = \{d,c,i\}$

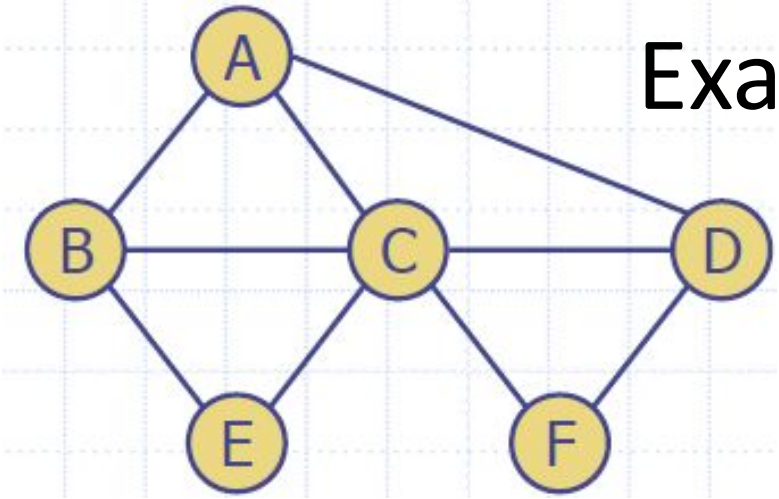


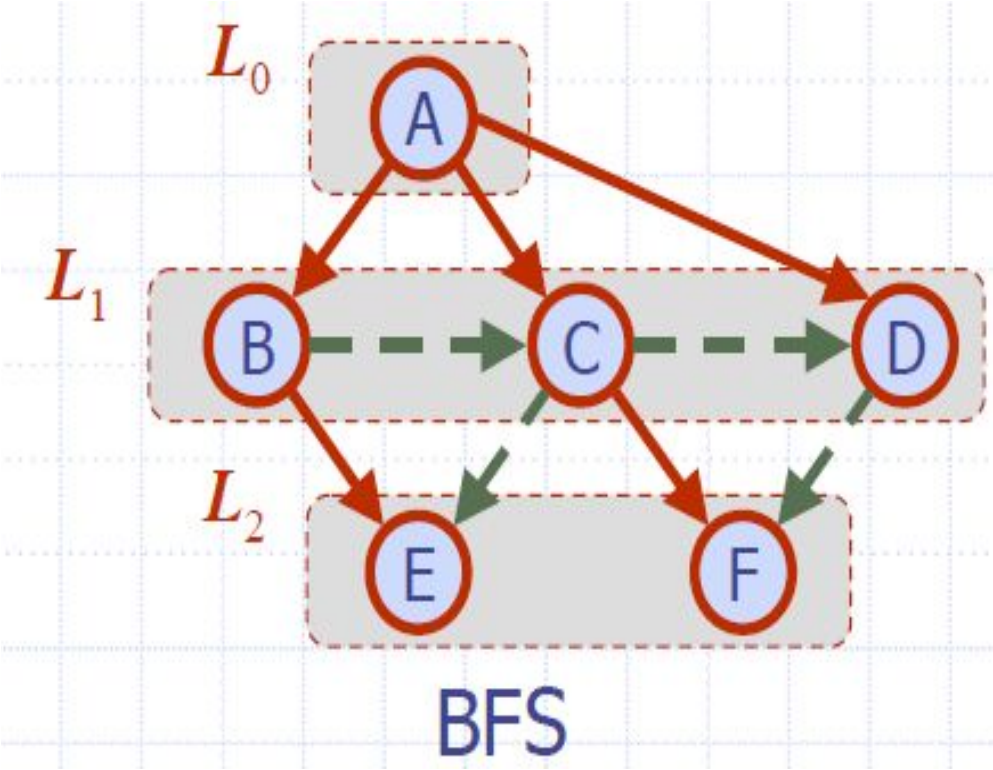
$Q = \{g,h\}$

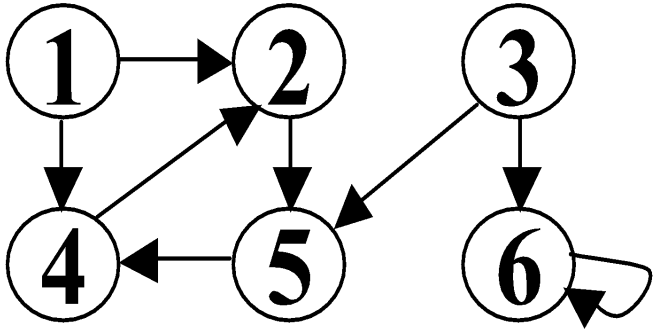


$Q = \{\}$

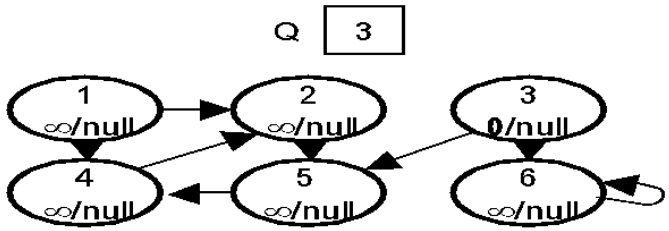
Example

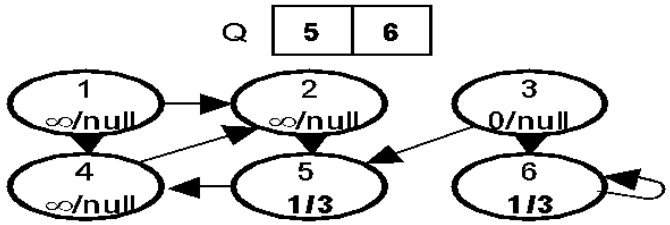


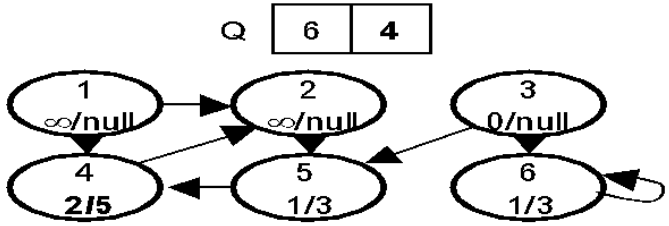


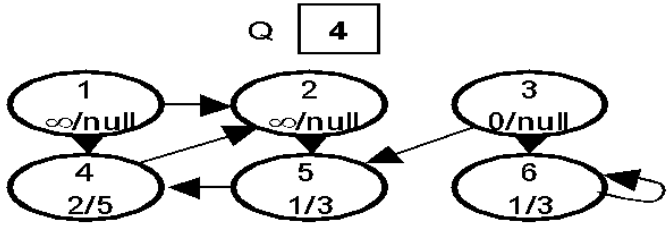

1
2
3
4
5
6

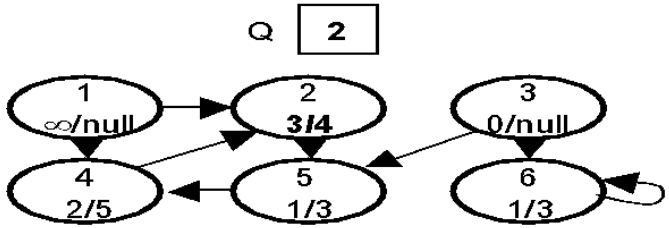
- ANS:

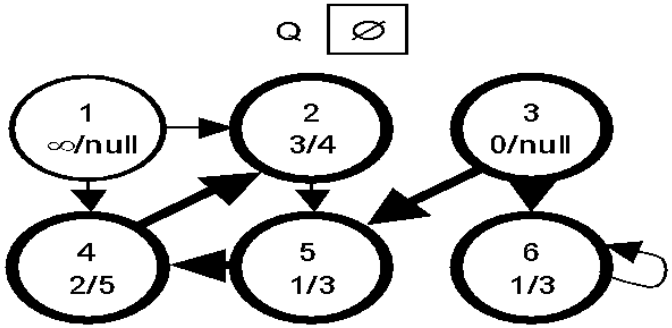

Q
3
1 ∞/null
2 ∞/null
3 0/null
4 ∞/null
5 ∞/null
6 ∞/null


Q
5
6
1 ∞/null
2 ∞/null
3 0/null
4 ∞/null
5 1/3
6 1/3


Q
6
4
1 ∞/null
2 ∞/null
3 0/null
4 2/5
5 1/3
6 1/3


Q
4
1 ∞/null
2 ∞/null
3 0/null
4 2/5
5 1/3
6 1/3


Q
2
1 ∞/null
2 3/4
3 0/null
4 2/5
5 1/3
6 1/3


Q
∅
1 ∞/null
2 3/4
3 0/null
4 2/5
5 1/3
6 1/3

Breadth First Search

- procedure bfsearch(G)

  {

  for each v ∈ N do mark[v] □ not-visited

  for each v ∈ N do

  if mark[v] ≠ visited then bfs(v)

  }
- Procedure bfs(v)

  {

  Q □ empty-queue

  mark[v] □ visited

  enqueue v into Q

  while Q is not empty do

  u □ first(Q)

  dequeue u from Q

  for each node w adjacent to u do

  if mark[w] ≠ visited

  then mark[w] □ visited

  enqueue w into Q

  }

Depth First Search - Undirected Graph

- Let G = <N, A> be an undirected graph all of whose nodes we wish to visit. Suppose it is somehow possible to mark a node to show it has already been visited.

- To carry out a depth-first traversal of the graph, choose any node v ∈ N as the starting point. Mark this node to show it has been visited.

- Next, if there is a node adjacent to v that has not yet been visited, choose this node as a new starting point and call the depth-first search procedure recursively.

Depth First Search - Undirected Graph

- On return from the recursive call, if there is another node adjacent to v that has not been visited, choose this node as the next starting point, call the procedure recursively once again, and finished.

- If there remain any nodes of G that have not been visited, chose any one of them as a new starting point, and call the procedure yet again.

- Continue thus until all the nodes of G are marked.

Depth First Search - Undirected Graph

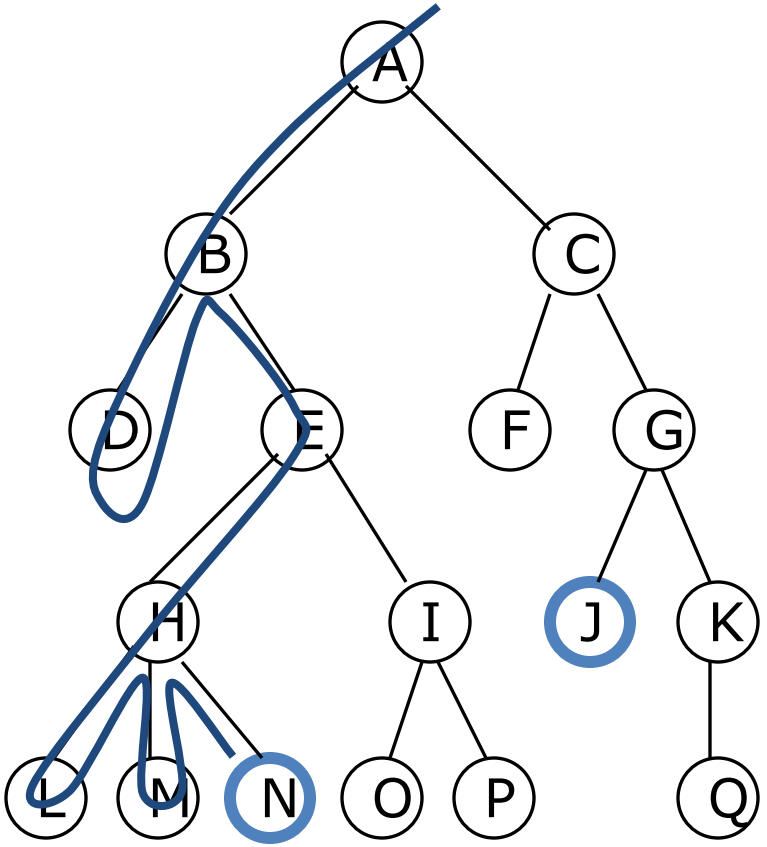
- procedure dfsearch(G)
  {
  for each v ∈ N do mark[v] □ not-visited
  for each v ∈ N do
  if mark[v] ≠ visited then dfs(v)
  }

- procedure dfs(v)
  {
  {Node v has not previously been visited}

  pnum □ pnum + 1
  prenum[v] □ pnum

  mark[v] □ visited
  for each node w adjacent to v do
  if mark[w] ≠ visited then dfs(w)
  }

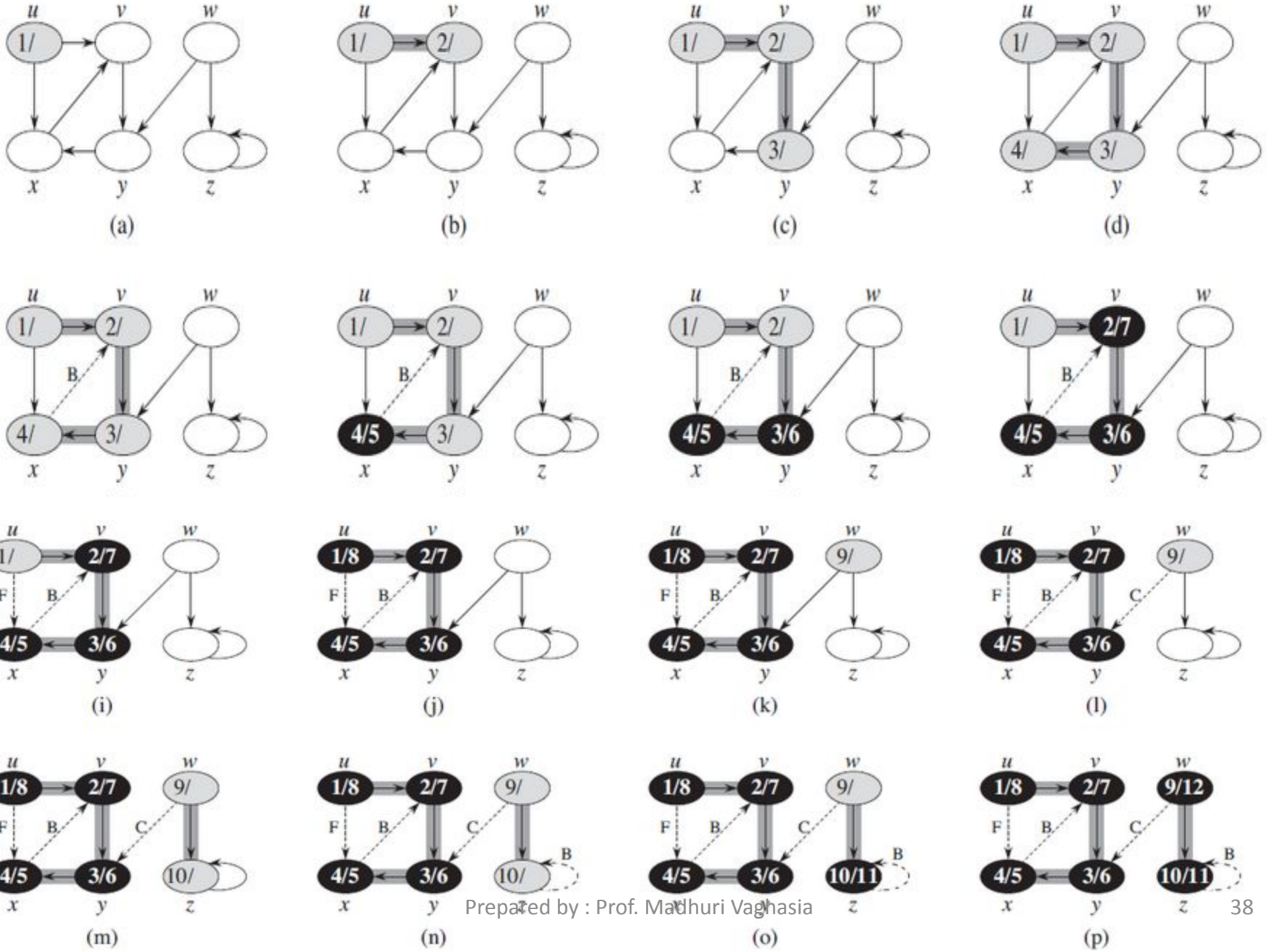
Depth-first searching



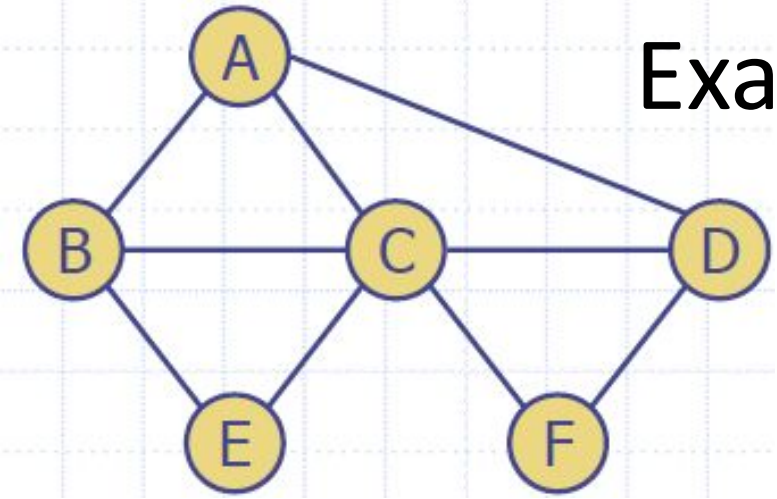
- A depth-first search (DFS) explores a path all the way to a leaf before backtracking and exploring another path
- For example, after searching A, then B, then D, the search backtracks and tries another path from B
- Node are explored in the order A B D E H L M N I O P C F G J K Q
- N will be found before J

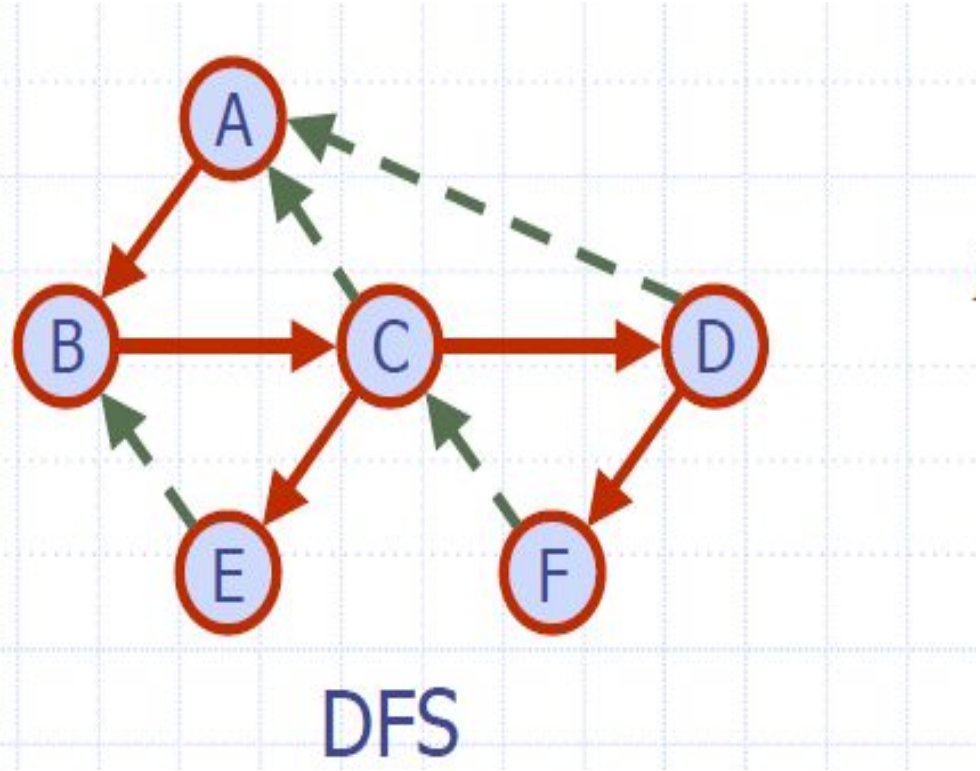
DFS-Procedure

- start from root
- travel to a leaf along a path
- backtrack one level
- travel down another branch
- if all branches of a node explored: backtrack another level
- continue until all sub trees of root explored
- appropriate data structure: **stack**

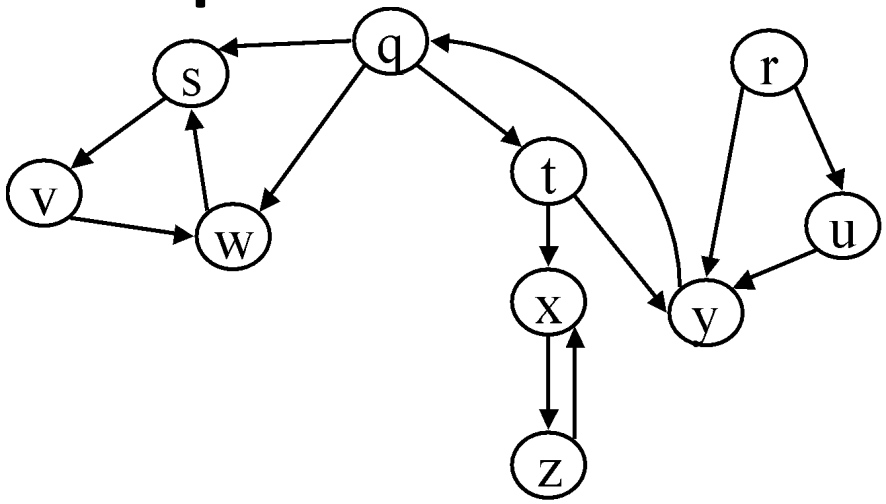

u v w
1/
x y z
(a)
u v w
1/ 2/
x y z
(b)
u v w
1/ 2/
3/
x y z
(c)
u v w
1/ 2/
4/ 3/
x y z
(d)
u v w
1/ 2/
B
4/ 3/
x y z
u v w
1/ 2/
B
4/5 3/
x y z
u v w
1/ 2/
B
4/5 3/6
x y z
u v w
1/ 2/7
B
4/5 3/6
x y z
u v w
1/ 2/7
F B
4/5 3/6
x y z
(i)
u v w
1/8 2/7
F B
4/5 3/6
x y z
(j)
u v w
1/8 2/7 9/
F B
4/5 3/6
x y z
(k)
u v w
1/8 2/7 9/
F B C
4/5 3/6
x y z
(l)
u v w
1/8 2/7 9/
F B C
4/5 3/6 10/
x y z
(m)
u v w
1/8 2/7 9/
F B C
4/5 3/6 10/ B
x y z
(n)
u v w
1/8 2/7 9/
F B C
4/5 3/6 10/11 B
x y z
(o)
u v w
1/8 2/7 9/12
F B C
4/5 3/6 10/11 B
x y z
(p)

Example

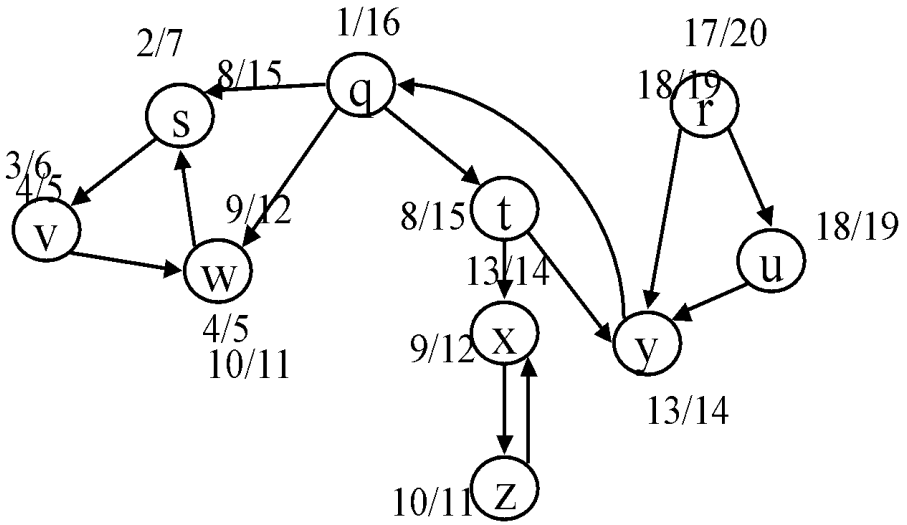




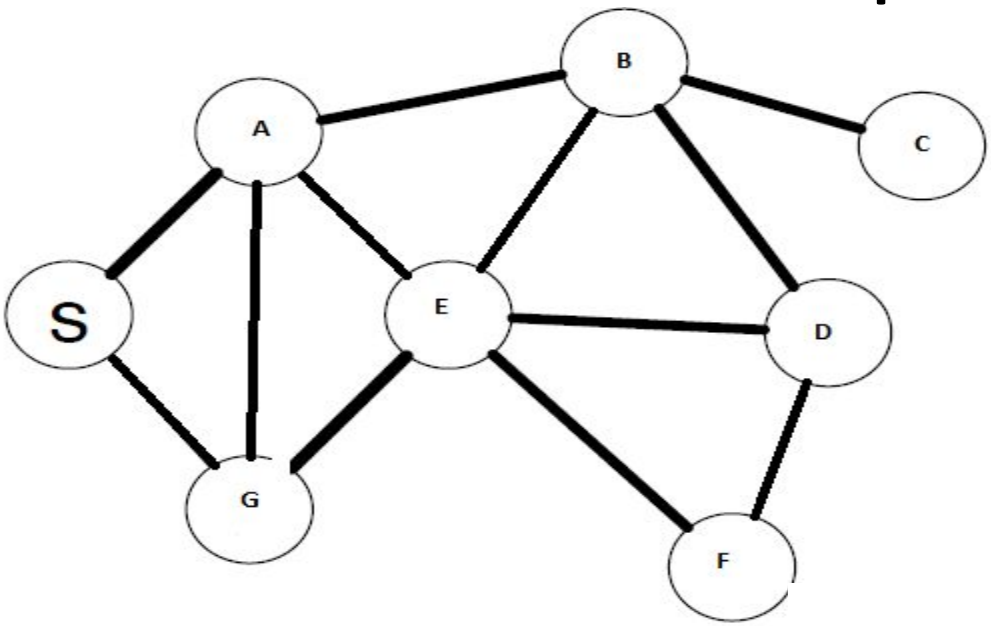
Example on DFS

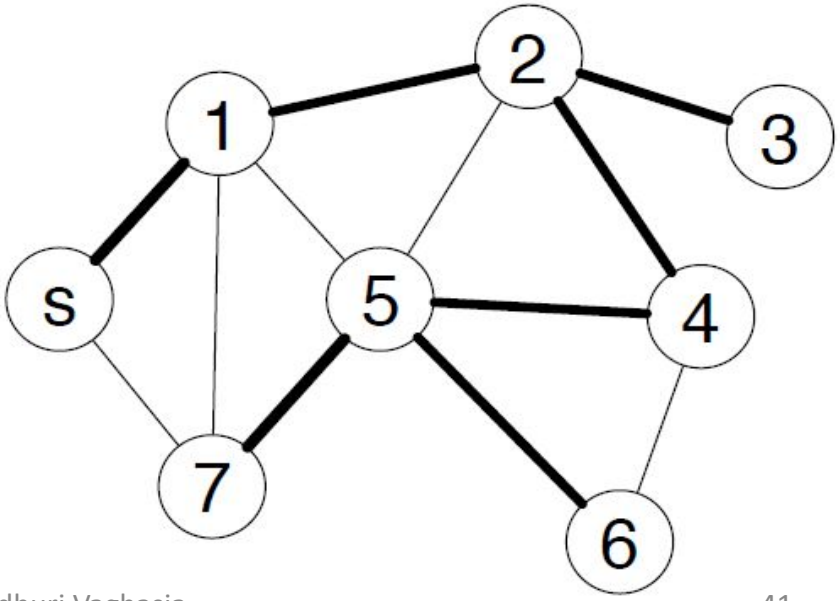


- ANS:



Example



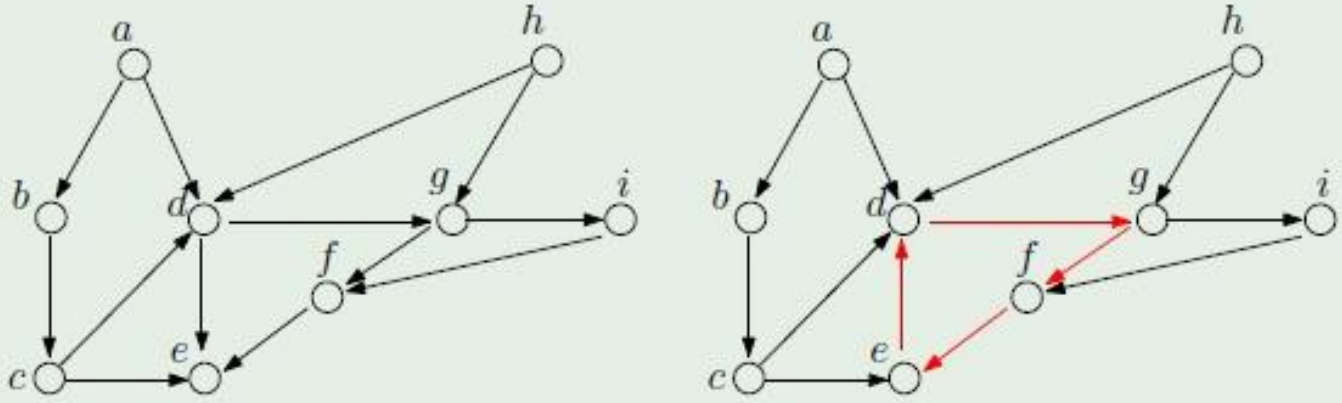


Topological sorting

Directed acyclic graph

A directed graph G is a *dag* (*directed acyclic graph*) if it does not have any cycle.

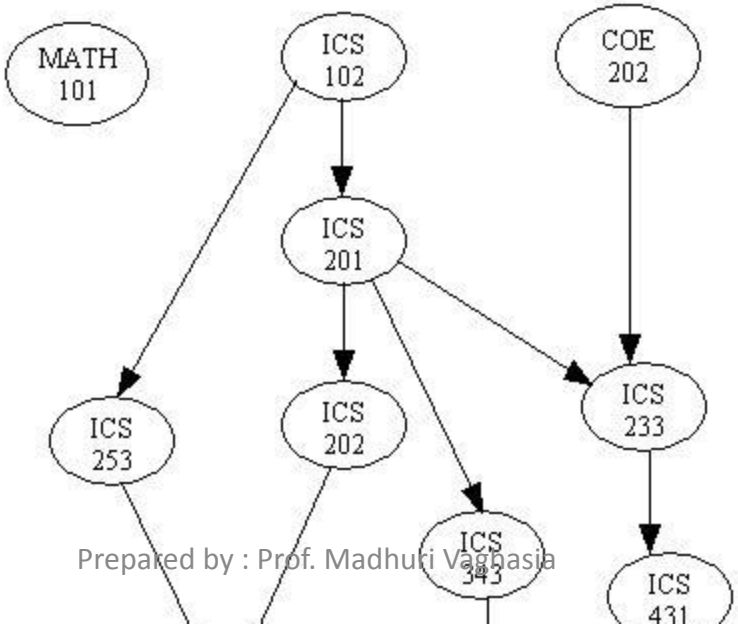
Example



The left graph is a dag, but the right is not.

Topological sorting

- There are many problems involving a set of tasks in which some of the tasks must be done before others.
- For example, consider the problem of taking a course only after taking its prerequisites.
- Is there any systematic way of linearly arranging the courses in the order that they should be taken?



Topological sorting

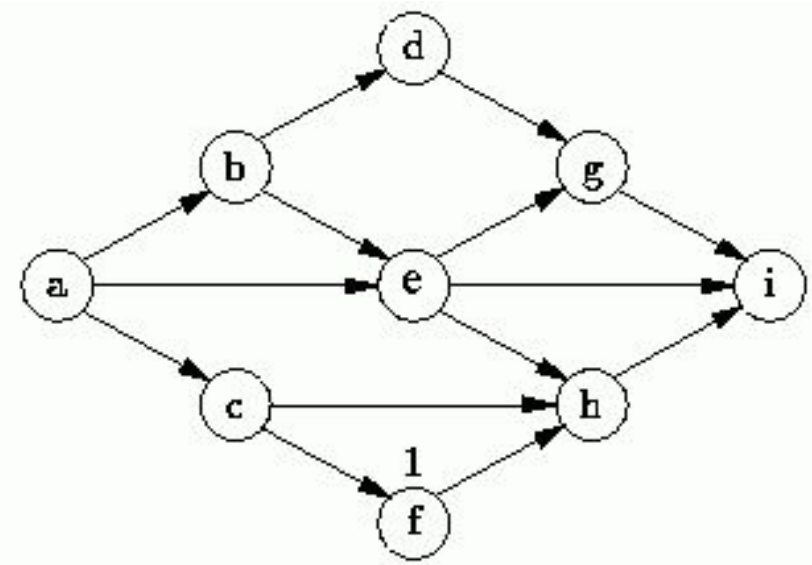
- The goal of a topological sort is given a list of items with dependencies, (ie. item 5 must be completed before item 3, etc.) to produce an ordering of the items that satisfies the given constraints. In order for the problem to be solvable, there can not be a cyclic set of constraints.

- We can model a situation like this using a directed acyclic graph. Given a set of items and constraints, we create the corresponding graph as follows:
  - 1) Each item corresponds to a vertex in the graph.
  - 2) For each constraint where item a must finish before item b, place a directed edge in the graph starting from the vertex for item a to the vertex for item b.

Topological sorting

- Topological sort is not unique.

- The following are all topological sort of the graph below:



s1 = {a, b, c, d, e, f, g, h, i}

s2 = {a, c, b, f, e, d, h, g, i}

s3 = {a, b, d, c, e, g, f, h, i}

s4 = {a, c, f, b, e, h, d, g, i}

etc.

Topological sorting

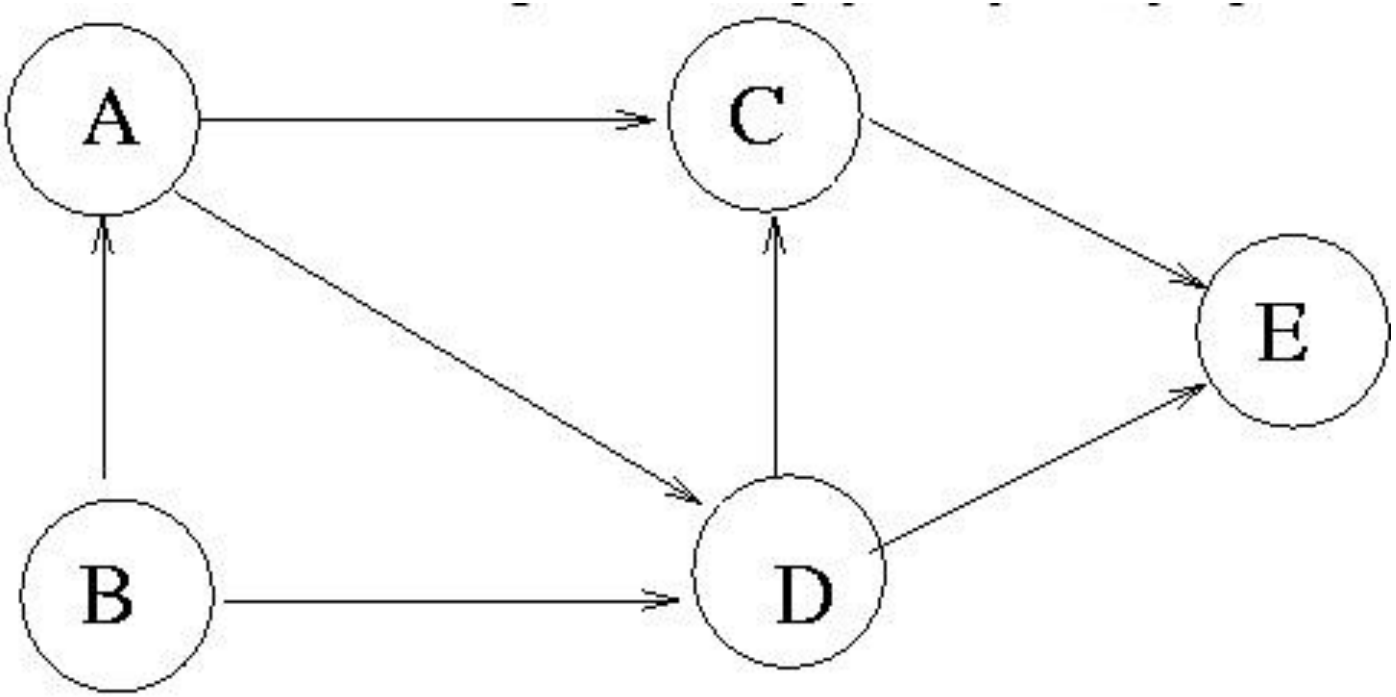
Topological Sorting; graphs

If $G = (V, E)$ is a DAG then a topological sorting of V is a linear ordering of V such that for each edge (u, v) in the DAG, u appears before v in the linear ordering.

Idea of Topological Sorting: Run the DFS on the DAG and output the vertices in reverse order of finishing time.

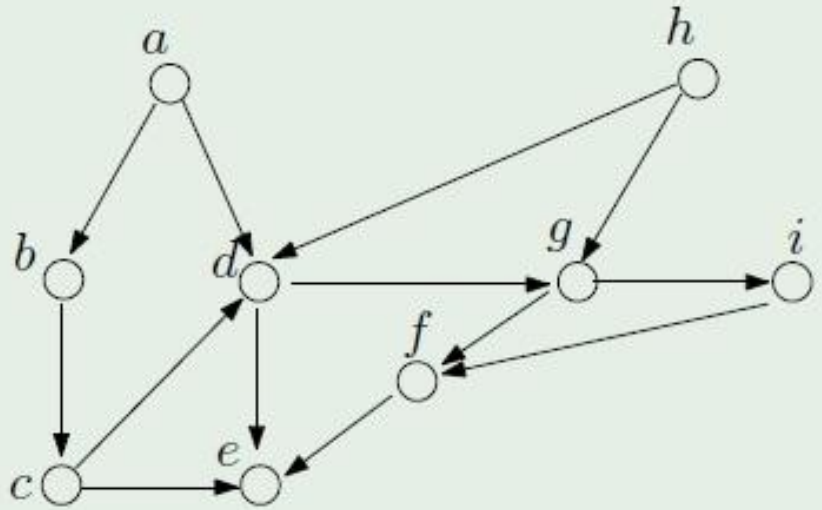
Topological sorting

- Example : A topological sort is given by: B, A, D, C, E.
- ->There could be several topological sorts for a gi



Topological sorting

Example



- A topological order: $a, b, c, h, d, g, i, f, e$.
- Another: $h, a, b, c, d, g, i, f, e$.

The result is not unique.

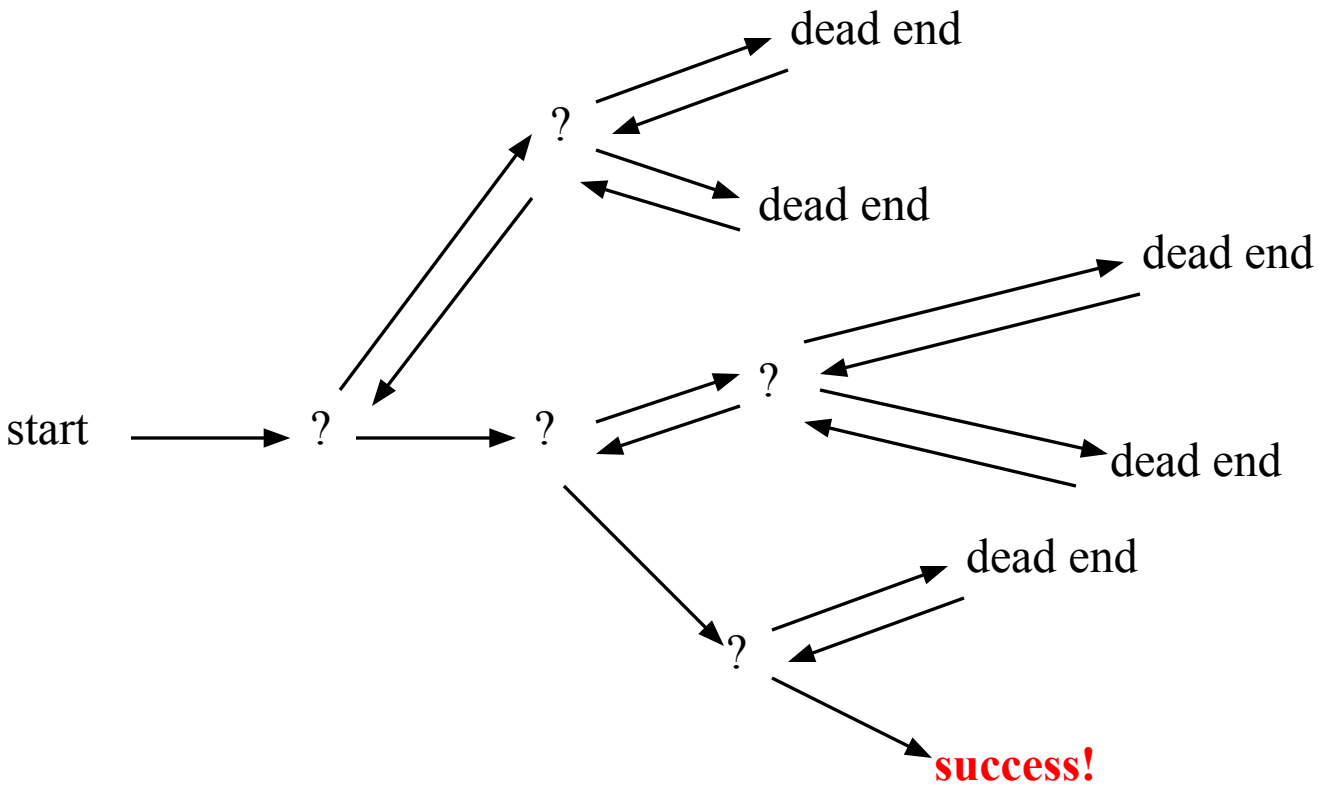
Backtracking

- Backtracking is kind of solving a problem by trial and error. However, it is a well organized trial and error. We make sure that we never try the same thing twice.
- We also make sure that if the problem is finite we will eventually try all possibilities (assuming there is enough computing power to try all possibilities).

Need for Backtracking

- Suppose you have to make a series of *decisions,* among various *choices,* where
  - You don't have enough information to know what to choose
  - Each decision leads to a new set of choices
  - Some sequence of choices (possibly more than one) may be a solution to your problem
- Backtracking is a methodical way of trying out various sequences of decisions, until you find one that "works"

Backtracking (animation)



Backtracking Algorithm

- Based on depth-first recursive search Approach
  1. Tests whether solution has been found
  2. If found solution, return it
  3. Else for each choice that can be made
     a) Make that choice
     b) Recur
     c) If recursion returns a solution, return it
  4. If no choices remain, return failure
- Some times called “search tree”

The general template

- Procedure backtrack(v[1…k])

  {

  { v is a k – promising vector}

  if v is a solution then write v

  {else} for each (k+1) – promising vector w

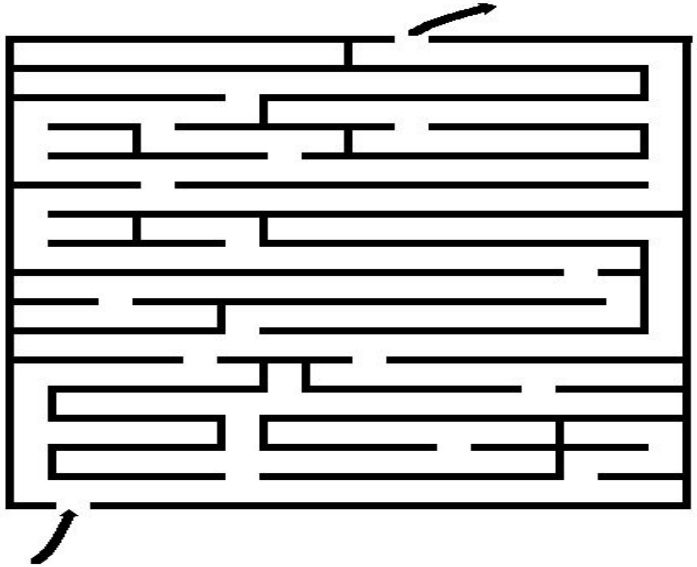
  such that w[1…k] = v[1…k]

  do backtrack(w[1…k+1])

  }

Backtracking Algorithm – Example

- Find path through maze
  - Start at beginning of maze
  - If at exit, return true
  - Else for each step from current location
    - Recursively find path
    - Return with first successful step
    - Return false if all steps fail
    - Other puzzles like 8-queen puzzle, Crosswords, Verbal Arithmetic, Sudoku, Peg solitaire,etc.



Sudoku

			2	6		7		1
6	8			7			9	
1	9				4	5		
8	2		1				4	
		4	6		2	9		
	5				3		2	8
		9	3				7	4
	4			5			3	6
7		3		1	8			

Sudoku

	6			1				2
	4		6	2		7		
			3		7		8	
	2			4		9		
		1				2		
		3		8			5	
	9		8		4			
		4		3	2		9	
2				7			4	

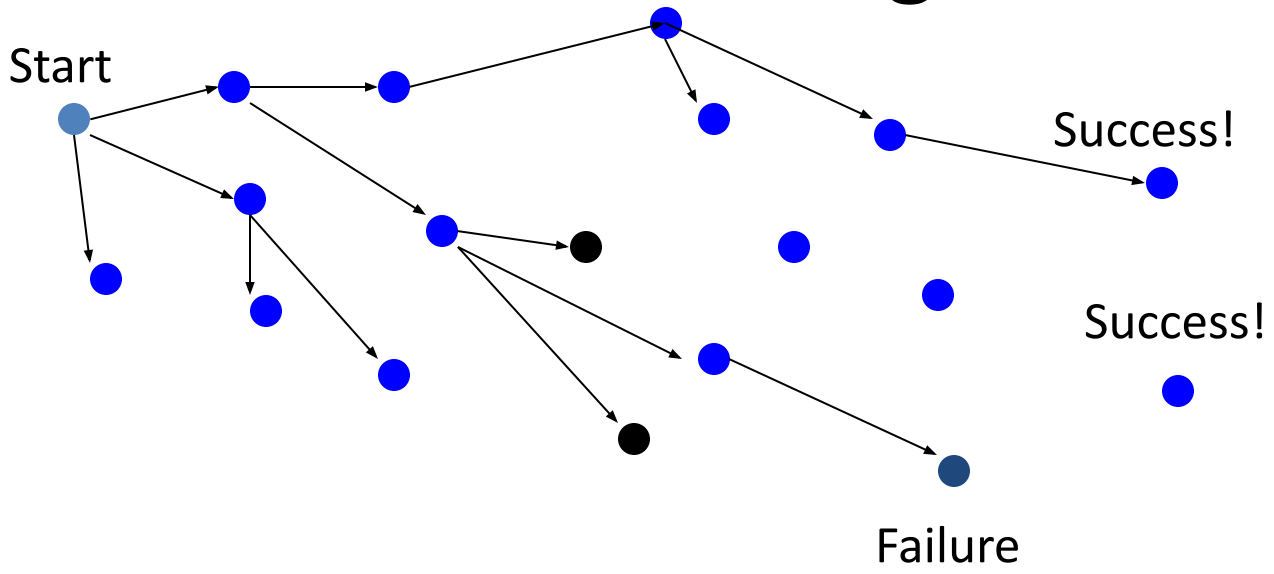
Sudoku

					3		8	5
		1		2				
			5		7			
		4				1		
	9							
5							7	3
		2		1				
				4				9

Backtracking Algorithm – Example

- Color a map with no more than four colors
  - If all countries have been colored return success
  - Else for each color c of four colors and country n
    - If country n is not adjacent to a country that has been colored c
      - Color country n with color c
      - Recursively color country n+1
      - If successful, return success
  - Return failure

Backtracking



Problem space consists of states (nodes) and actions (paths that lead to new states). When in a node can can only see paths to connected nodes

If a node only leads to failure go back to its "parent" node. Try other alternatives. If these all lead to failure then more backtracking may be necessary.

Recursive Backtracking

Pseudo code for recursive backtracking algorithms

If at a solution, return success
for(every possible choice from current state / node)
Make that choice and take one step along path
Use recursion to solve the problem for the new node / state
If the recursive call succeeds, report the success to the next high level
Back out of the current choice to restore the state at the beginning of the loop.
Report failure

Backtracking

- Construct the state space tree:
  - Root represents an initial state
  - Nodes reflect specific choices made for a solution's components.
    - Promising and nonpromising nodes
    - leaves

- Explore the state space tree using depth-first search

- "Prune" non-promising nodes
  - dfs stops exploring subtree rooted at nodes leading to no solutions and...
  - "backtracks" to its parent node

0/1 Knapsack Problem using Backtracking

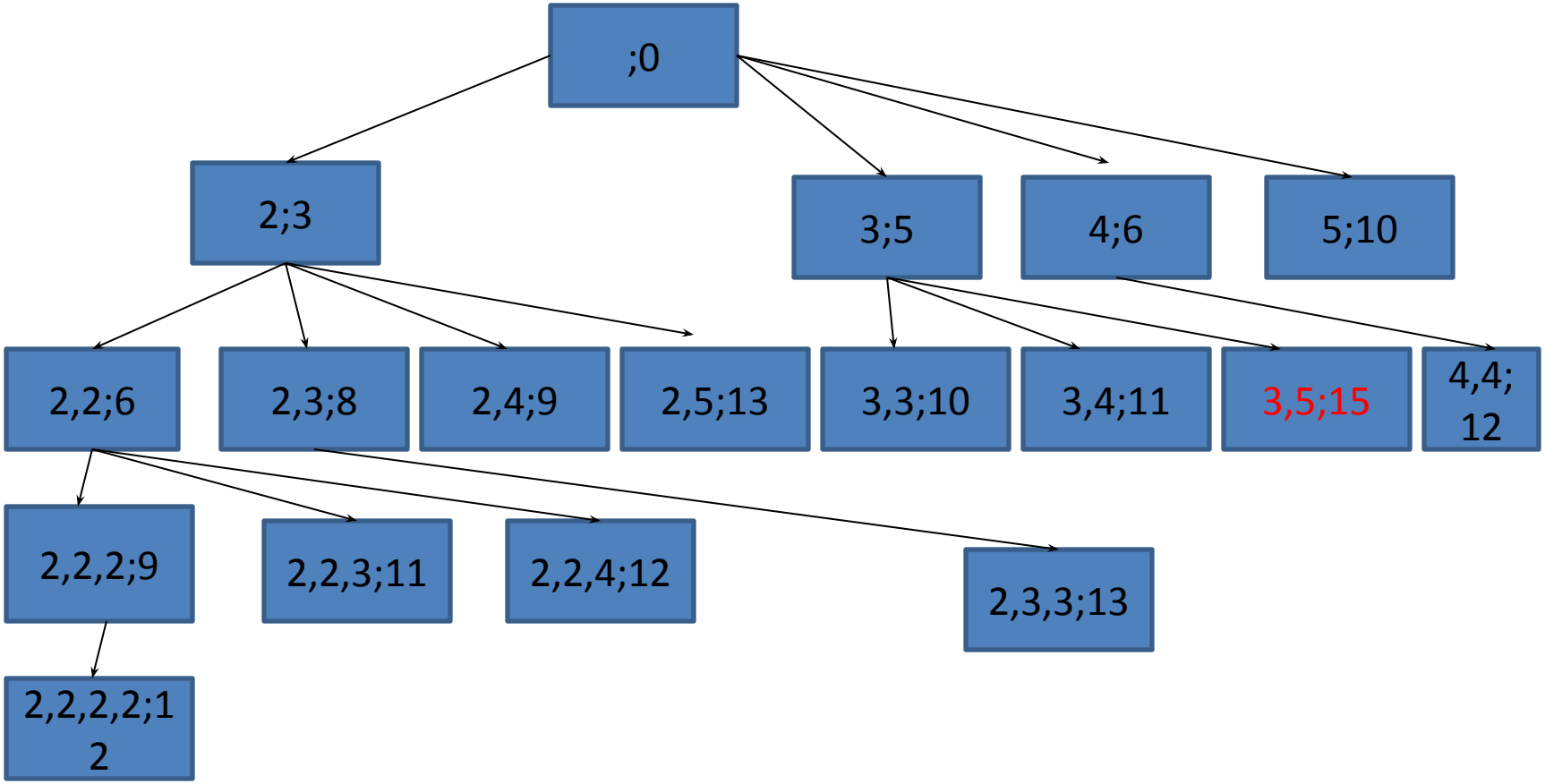
- Knapsack can carry a weight not exceeding W(total knapsack capacity)
- Main Aim: To fill the knapsack in a way the maximize the profit of the included objects, while respecting the capacity constraint.
- We may take an object or to leave it behind, but we may not take a fraction of an object

Variations on knapsack problem

- **Fractions are allowed. This applies to items such as:**
  - **bread, for which taking half a loaf makes sense**
  - **gold dust**
- **No fractions.**
  - **0/1 (1 brown pants, 1 green shirt…)**
  - **Allows putting many items of same type in knapsack**
    - **5 pairs of socks**
    - **10 gold bricks**
  - **More than one knapsack, etc.**

An Example:

Weight	2	3	4	5
Profit	3	5	6	10
Capacity	8			



Weight	2	3	4	5
Profit	3	5	6	10
Capacity	8			

Explanation

- (2,3;8) -> 2 & 3 are the weights included and 8 is the profit earned
- Decreasing weight also can be used, just it reduces the size of the tree to be searched.
- Once we have visited (2,2,3;11) there is no point in later visiting (2,3,2;11)
- Initially the partial solution is empty.
- The backtracking algorithm explores the tree as in DFS (memorization), constructing nodes and partial solution as it goes.

Algorithm

```
function backtrack(i , r)
{ Calculates the value of the best load that can be
    constructed using items of types I to n and whose
    total weight does not exceed r }
b = 0
{ Try each allowed kind of item in turn }
for k = i to n do
{
    if (w[k]<=r) then
     b = max(b , v[k]+ backpack (k, r-w[k]))
}
return b;
```

- Backtrack(1,8) i=1 r=8(maximum weight)

K=1 to 4

W[1] < 8 i.e 2 < 8 yes

b= max(0,3+BT(1,8-2))

b= max(0,3+BT(1,6))

Algorithm Explanation

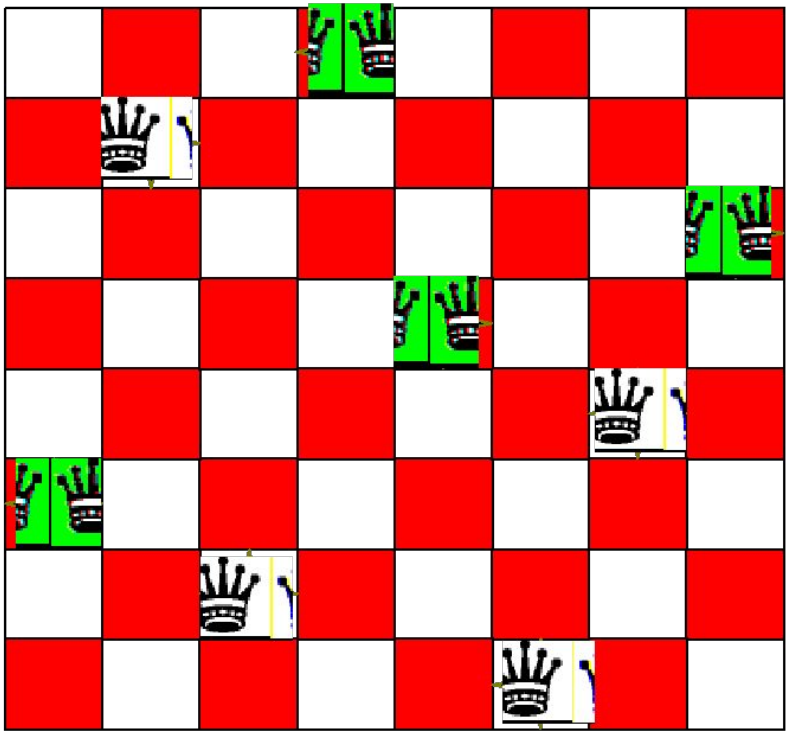
- Initially call backpack(1,w)
- Each recursive call of backpack corresponds to extending the depth first search one level down the tree, while the for loop takes care of examining all the possibilities at a given level.

N-queen Problem

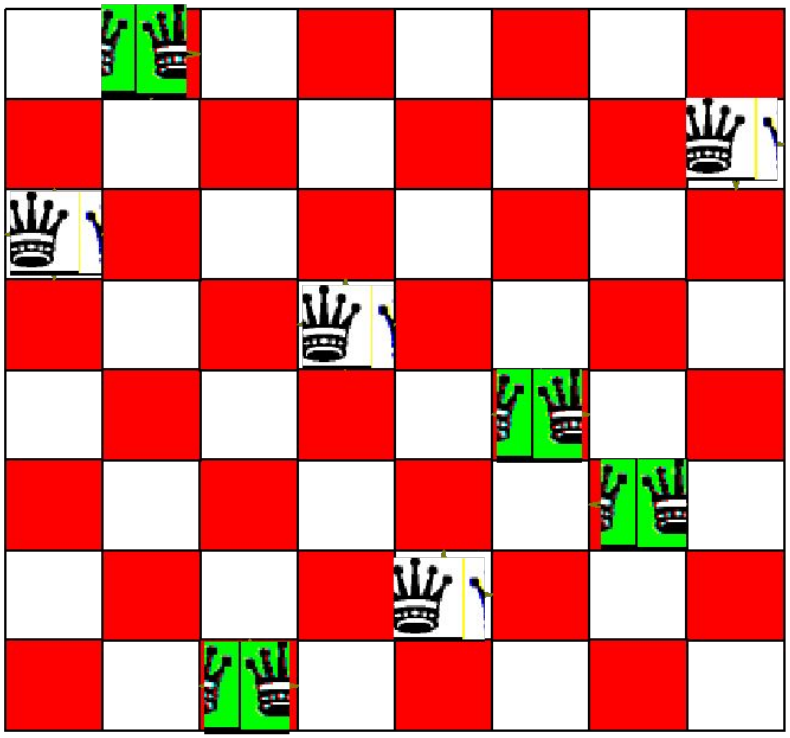
- Given is a chess board. A chess board has 8x8 fields. Is it possible to place 8 queens on this board, so that no two queens can attack each other?
- A queen can attack horizontally, vertically, and on both diagonals, so it is pretty hard to place several queens on one board so that they don’t attack each other.

Example: The *n*-Queen problem

- Place *n* queens on an *n* by *n* chess board so that no two of them are on the same row, column, or diagonal

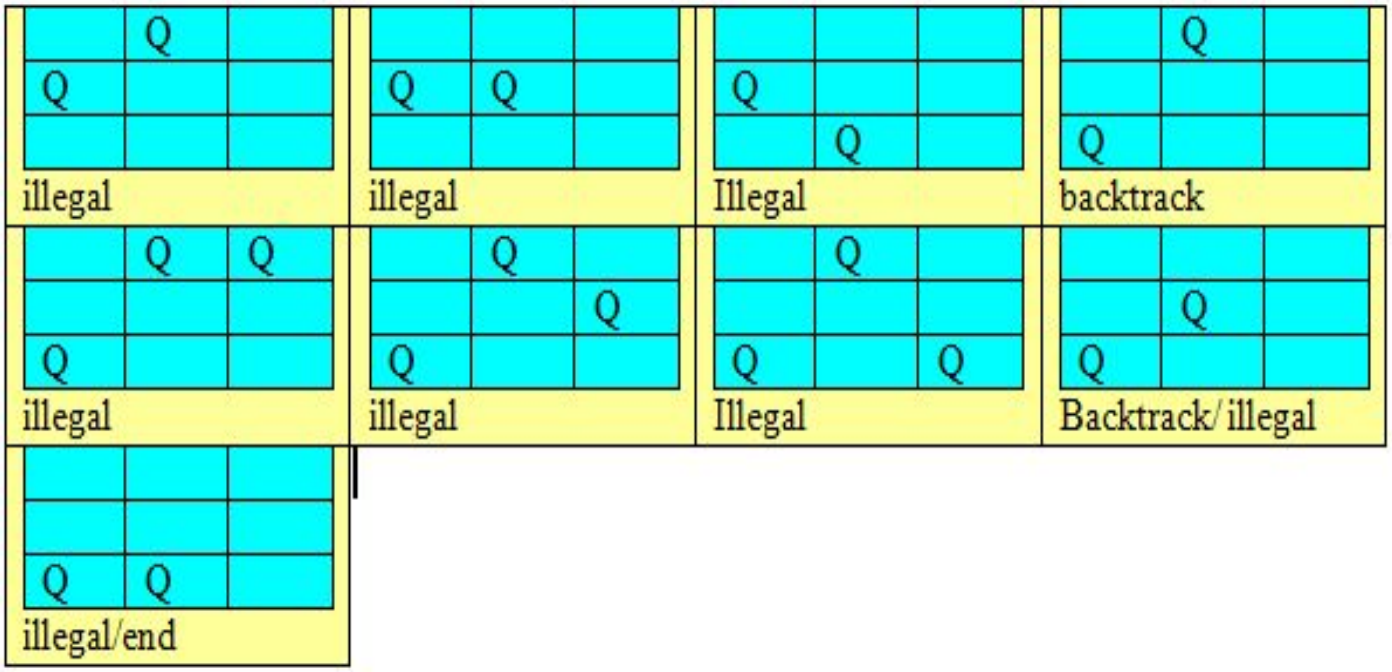


Solution



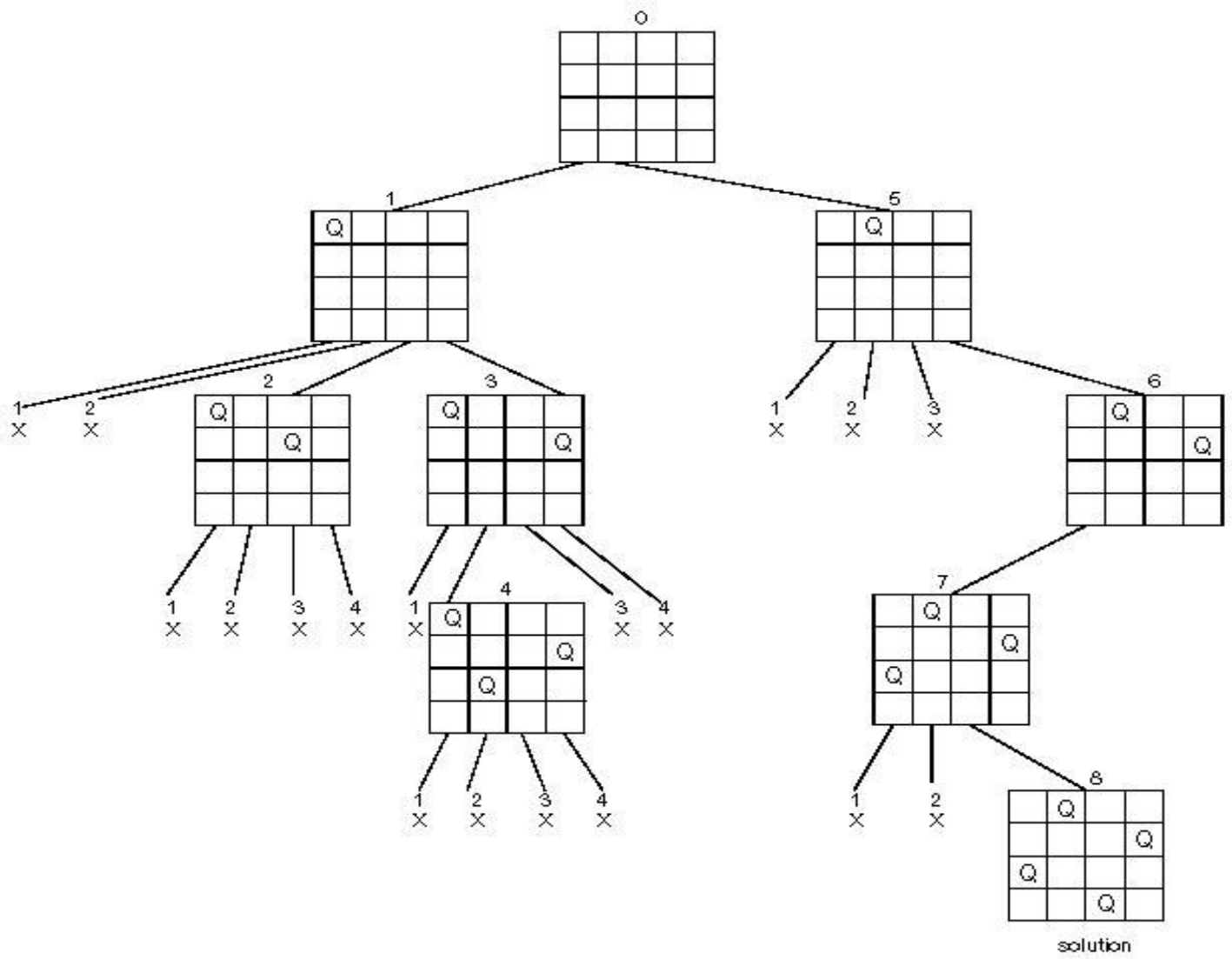
Not a Solution

Solving a 3-queen Problem

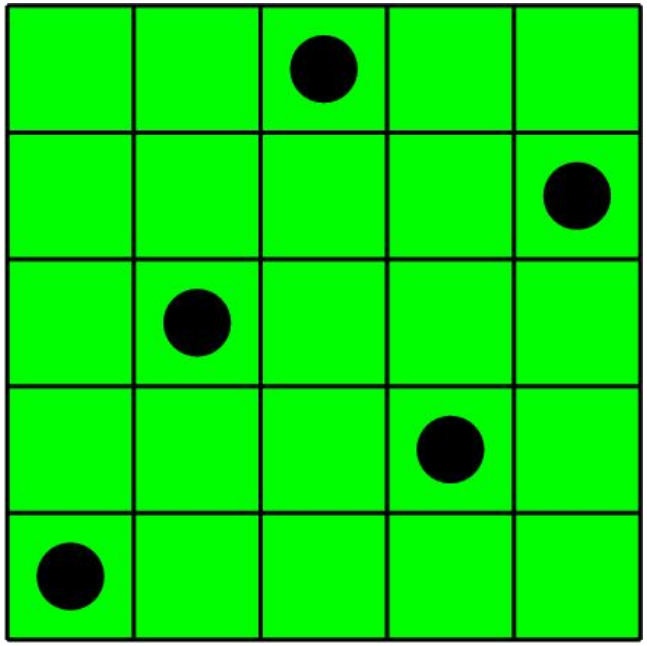


Solving 4-queen problem

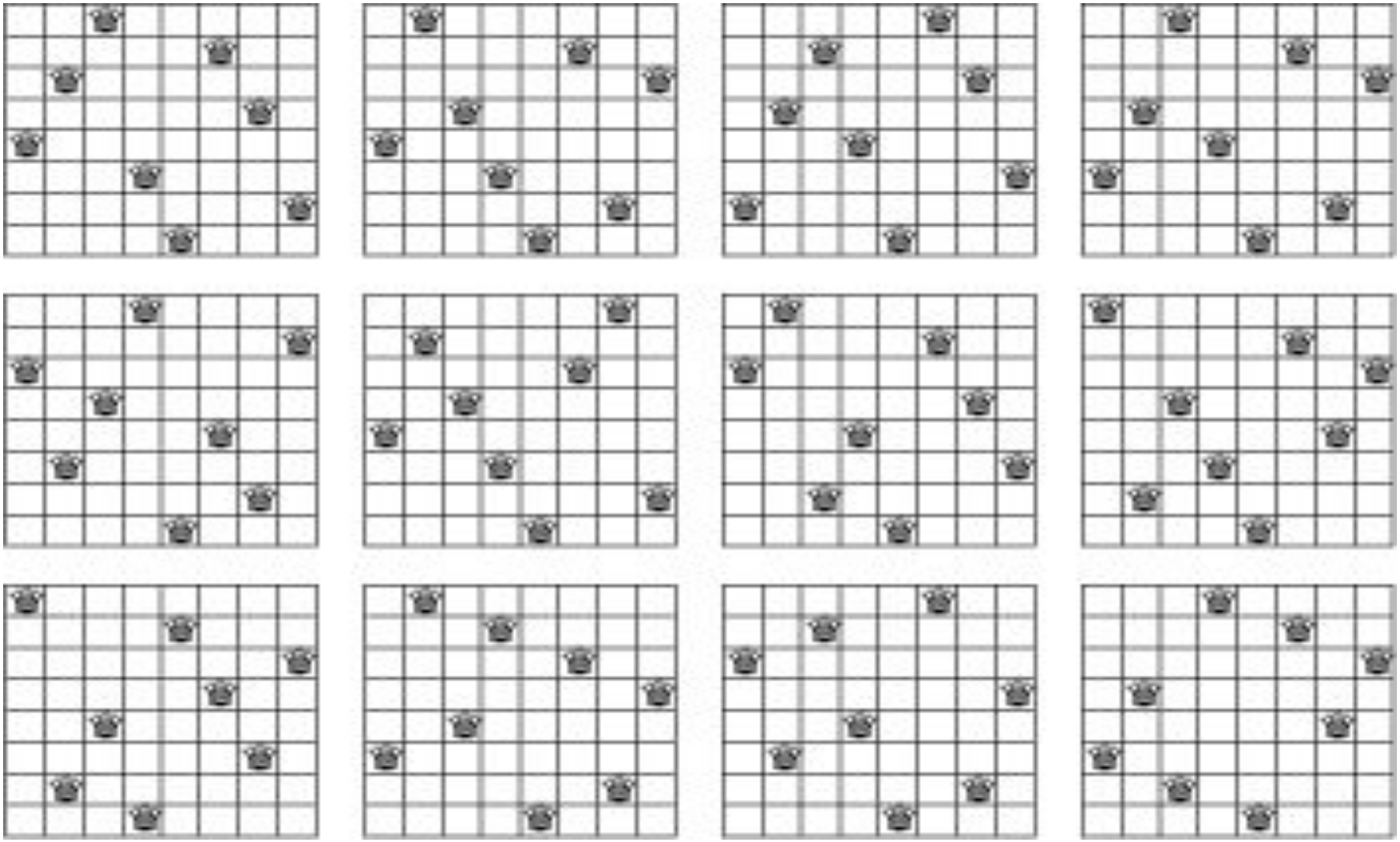
State Space Tree of the Four-queens Problem



Solution to 5-queen Problem



Some solution to 8 queen problem



```
Algorithm Place(k,i)
/* returns true if a queen can be placed in kth row and ith
   column. Otherwise it returns false. X[ ] is a gloabal array
   whose first (k-1) values have been set. Abs( r) returns the
   absolute value of r. */
{
for j:=1 to k-1 do
   if((x[j]=i) /* two in the same column */
     or (Abs(x[j]-i)=Abs(j-k))) /* or in the same daigonal */
     then return false;
return true;
}
```

```
Algorithm Queens(k,8)
/* using backtracking ,this procedure pritns all possible placement of n
    queens on 8×8 chessboard so that they are nonattacking  */
{
    for i:=1 to 8 do
    {
     if(Place(k,i) then
      {
       x[k]:=i;
       if(k=8) then write(x[1..8]);
       else Queens(k+1,8);
        }
    }
}
```

GTU QUESTIONS..

- Explain in brief Breadth First Search and Depth First Search Traversal techniques of a Graph. [7]
- Find all possible solution for the 4*4 chessboard, 4 queen's problem using backtracking. [7]
- Define: Acyclic Directed Graph, Articulation Point, Dense Graph, Sparse Graph. [4]

PG

- For a graph G = (V, E) with V no of Vertices and E no of Edges. Where V=12 and E=23. What kind of storage would be used to store Information of Adjacent Nodes? What would be storage requirement/complexity for maintaining information for adjacent List and adjacent Matrix?
- Explain different types of Edges in DFS with suitable example.
- Find an optimal solution to the knapsack instance n=4, M=8, (P1,P2,P3,P4)= (3,5,6,10) and (W1,W2,W3,W4)=(2,3,4,5) using backtracking.